AI VIET NAM
@aivietnam.edu.vn

NGUYÊN NHÂN VÀ CƠ CHẾ BIẾN ĐỔI GRADIENT TRONG MẠNG NƠ-RON SÂU

Hồ Văn Lực **Phùng Đức Tuấn Kiệt**

Proofreading: AI VIET NAM Mentorship Team

Mục lục

I. Tổng quan: nghịch lý của chiều sâu . . . 3

I.1. Hai chế độ suy biến . . . 3

I.2. Cái bẫy mạng nông . . . 4

I.3. Cấu trúc tài liệu . . . 4

II. Cơ sở toán học của lan truyền ngược . . . 4

II.1. Quy ước ký hiệu . . . 4

II.2. Jacobian giữa hai tầng liên tiếp . . . 5

II.3. Công thức đệ quy của tín hiệu lỗi . . . 5

II.4. Đối tượng trung tâm: tích Jacobian . . . 6

III. Hai nguyên nhân cốt lõi . . . 7

III.1. Nguyên nhân 1 — độ bão hoà của hàm kích hoạt . . . 7

III.1.1. Hàm tanh làm chậm vấn đề nhưng không loại bỏ nó . . . 8

III.2. Nguyên nhân 2 — bán kính phổ của ma trận trọng số . . . 9

III.3. Hợp nhất: Chặn trên bảo thủ của hệ số co giãn Jacobian . . . 9

III.4. Nghịch lý phương sai: vì sao không thể phóng đại trọng số . . . 10

III.4.1. Truy hồi phương sai theo chiều tiến . . . 10

III.4.2. Thế kẹt . . . 11

III.5. Dynamical Isometry: phổ của toàn bộ Jacobian . . . 12

IV. Minh hoạ số học: chuỗi ba tầng . . . 14

IV.1. Lan truyền tiến . . . 14

IV.2. Lan truyền ngược, từng phép nhân một . . . 14
IV.3. Đọc kết quả . . . 15
IV.4. Cross-entropy có cứu được không? . . . 15
IV.4.1. Triệt tiêu ở tầng ra . . . 16
IV.4.2. Tính lại với cùng bộ số . . . 16
IV.4.3. Kết quả quyết định: tỉ lệ suy giảm không phụ thuộc hàm mất mát . . . 17

V. Lan truyền ngược theo thời gian . . . 17

V.1. Mô hình và ký hiệu . . . 18
V.2. Công thức đệ quy . . . 18
V.3. Vì sao RNN tệ hơn MLP cùng độ sâu . . . 18
V.4. Phân tích phổ: điều gì thực sự quyết định tốc độ suy giảm . . . 19
V.4.1. Hai điều kiện cần nói rõ . . . 19
V.5. Quên thảm hoạ các phụ thuộc xa . . . 20
V.6. Băng chuyền sai số hằng: suy diễn thay vì khẳng định . . . 20
V.6.1. Vì sao RNN thuần không thể làm được điều tương tự . . . 21

VI. Thực nghiệm: đo chuẩn gradient theo độ sâu tại bước khởi tạo . . . 21

VI.1. Giao thức . . . 22
VI.2. Kết quả . . . 25
VI.3. Phân tích . . . 26
VI.4. Phổ Jacobian đầu-cuối tại khởi tạo . . . 26
VI.5. Động lực học trong quá trình huấn luyện: PlainNet so với ResNet . . . 28
VI.6. Hạn chế của nghiên cứu . . . 34

VII. Tổng kết và các nguyên lý khắc phục . . . 35

VII.1. Bốn nguyên lý, đọc qua lăng kính của c . . . 36
VII.2. Liên hệ với tài liệu Module 4 . . . 37

A. Hình bổ sung: $\log_{10}\rho$ theo độ sâu tại bước khởi tạo . . . 39

I. Tổng quan: nghịch lý của chiều sâu

Lý thuyết xấp xỉ phổ quát khẳng định một mạng nơ-ron đủ rộng có thể xấp xỉ mọi hàm liên tục trên tập compact với sai số tuỳ ý. Bổ sung thêm chiều sâu chỉ làm lớp hàm biểu diễn được *giàu hơn*: một mạng $L+1$ tầng luôn có thể mô phỏng mạng L tầng bằng cách đặt tầng thừa thành ánh xạ đồng nhất. Vì vậy, theo lý thuyết, mạng sâu hơn không bao giờ được phép tệ hơn.

Thực nghiệm những năm 1990–2000 lại cho kết quả ngược lại. Khi tăng số tầng của một mạng kết nối đầy đủ dùng hàm kích hoạt sigmoid từ 5 lên 20 tầng, *sai số huấn luyện* — chứ không phải sai số kiểm tra — tăng lên. Đây là điểm mấu chốt cần tách bạch ngay từ đầu:

- Nếu sai số huấn luyện thấp nhưng sai số kiểm tra cao, ta gặp vấn đề **tổng quát hoá** (quá khớp). Cách chữa là chính quy hoá, thêm dữ liệu, giảm dung lượng mô hình.
- Nếu **sai số huấn luyện** tự nó không giảm được, mô hình thậm chí không học nổi dữ liệu nó đang nhìn thấy. Đây là vấn đề **tối ưu hoá**. Thêm dữ liệu hay chính quy hoá đều vô ích.

Hiện tượng triệt tiêu gradient thuộc loại thứ hai. Mô hình có đủ dung lượng biểu diễn, hàm mục tiêu có tồn tại nghiệm tốt, nhưng thuật toán hạ gradient không truyền được tín hiệu học tới các tầng đầu. Nói cách khác, *không gian giả thuyết chứa nghiệm nhưng quỹ đạo tối ưu không tới được nó.*

Phát biểu vấn đề trong một câu

Trong mạng nơ-ron sâu, gradient của hàm mất mát theo tham số của tầng thứ l tỉ lệ với một *tích* gồm $L-l$ thừa số ma trận; nếu độ lớn đặc trưng của mỗi thừa số lệch khỏi 1, tích đó suy giảm hoặc bùng nổ theo cấp số nhân, khiến các tầng gần đầu vào hoặc không nhận được tín hiệu học, hoặc nhận tín hiệu vô nghĩa về mặt số học.

I.1. Hai chế độ suy biến

Gọi $\boldsymbol{g}^{[l]} = \partial \mathcal{L} / \partial \mathbf{W}^{[l]}$ là gradient tại tầng l. Hai chế độ hỏng hóc là đối xứng nhau:

Triệt tiêu gradient (vanishing gradient). $\|\boldsymbol{g}^{[l]}\| \to 0$ theo cấp số nhân khi l giảm về phía đầu vào. Biểu hiện thực tế: mất mát giảm rất nhanh trong vài epoch đầu rồi đứng yên ở một mức cao; kiểm tra trọng số cho thấy các tầng cuối thay đổi rõ rệt còn các tầng đầu gần như giữ nguyên giá trị khởi tạo. Đây là trường hợp chính của tài liệu này, và là trường hợp *nguy hiểm hơn* vì nó im lặng: không có cảnh báo, không có `NaN`, chỉ có một mô hình học kém mà ta dễ quy nhầm cho thiếu dữ liệu.

Bùng nổ gradient (exploding gradient). $\|\boldsymbol{g}^{[l]}\| \to \infty$ theo cấp số nhân. Biểu hiện: mất mát nhảy vọt rồi trở thành `NaN` hoặc `Inf` sau vài bước cập nhật. Chế độ này ồn ào nên dễ phát hiện, và có một cách chữa cơ học trực tiếp là cắt ngưỡng gradient *(gradient clipping)* [1].

Điểm sẽ được chứng minh ở Mục III. là hai hiện tượng này **không phải hai lỗi khác nhau**. Chúng là cùng một cơ chế — tích ma trận lặp — quan sát ở hai phía của ngưỡng 1.

I.2. Cái bẫy mạng nông

Hệ quả lịch sử của hiện tượng này là điều có thể gọi là *cái bẫy mạng nông*. Khi mạng sâu huấn luyện kém hơn mạng nông, kết luận tự nhiên — và sai — là "chiều sâu không giúp ích cho bài toán này". Người thực hành bèn quay lại kiến trúc 2–3 tầng, thấy nó hoạt động ổn định hơn, và củng cố niềm tin sai lầm đó. Suốt gần hai thập kỷ, cộng đồng đã đọc một *thất bại của thuật toán tối ưu* thành một *giới hạn của kiến trúc*.

Chẩn đoán đúng đến từ luận văn của Hochreiter [2] và phân tích hình thức của Bengio, Simard và Frasconi [3]: vấn đề nằm ở cấu trúc nhân của lan truyền ngược, không nằm ở chiều sâu. Khi nguyên nhân được hiểu đúng, các biện pháp khắc phục — khởi tạo có kiểm soát phương sai [4, 5], hàm kích hoạt không bão hoà [5], chuẩn hoá theo lô [6] và kết nối tắt [7] — đã cho phép huấn luyện mạng hàng trăm tầng.

Lưu ý

Trước khi kết luận "mô hình của tôi cần thêm dữ liệu", hãy kiểm tra sai số *huấn luyện*. Nếu chính nó không giảm, dữ liệu không phải là biến số cần điều chỉnh.

I.3. Cấu trúc tài liệu

Mục II. thiết lập ký hiệu và suy diễn đầy đủ công thức lan truyền ngược dạng ma trận. Mục III. tách vấn đề thành hai nguyên nhân độc lập — độ bão hoà của hàm kích hoạt và bán kính phổ của ma trận trọng số — rồi hợp nhất chúng thành một đại lượng điều khiển duy nhất. Mục IV. kiểm chứng bằng một ví dụ số học ba tầng tính tay. Mục V. mở rộng sang mạng hồi quy, nơi vấn đề nghiêm trọng hơn hẳn. Mục VI. đo đạc thực nghiệm trên mạng 10 tầng. Mục VII. tổng kết các nguyên lý khắc phục.

II. Cơ sở toán học của lan truyền ngược

II.1. Quy ước ký hiệu

Toàn bộ tài liệu tuân thủ nghiêm ngặt quy ước sau. Mọi kết luận ở các mục sau chỉ đúng khi quy ước này được giữ nguyên, đặc biệt là vị trí của các phép chuyển vị.

- **Gradient là vector cột** (bố cục mẫu số, *denominator layout*). Với $f : \mathbb{R}^n \to \mathbb{R}$, ta có $\partial f/\partial \mathbf{x} \in \mathbb{R}^{n\times 1}$.
- Mạng có L tầng. Tầng l có n_l nơ-ron; $\mathbf{W}^{[l]} \in \mathbb{R}^{n_l \times n_{l-1}}$ và $\mathbf{b}^{[l]} \in \mathbb{R}^{n_l}$.
- Lan truyền tiến:

$$\mathbf{z}^{[l]} = \mathbf{W}^{[l]}\mathbf{a}^{[l-1]} + \mathbf{b}^{[l]}, \qquad \mathbf{a}^{[l]} = \sigma\left(\mathbf{z}^{[l]}\right), \qquad \mathbf{a}^{[0]} = \mathbf{x}. \tag{1}$$

  Hàm σ tác động theo từng phần tử.

- Tín hiệu lỗi tại tầng l:

$$\boldsymbol{\delta}^{[l]} := \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{[l]}} \in \mathbb{R}^{n_l}. \tag{2}$$

Lưu ý

$\boldsymbol{\delta}$ được định nghĩa theo *tiền kích hoạt* $\mathbf{z}$, không phải theo $\mathbf{a}$. Đây là lựa chọn có chủ đích: nó khiến ma trận đường chéo chứa σ' xuất hiện ở bên trái trong công thức đệ quy, và làm lộ rõ cấu trúc tích mà toàn bộ tài liệu xoay quanh.

II.2. Jacobian giữa hai tầng liên tiếp

Từ (1), viết $\mathbf{z}^{[k]}$ như hàm của $\mathbf{z}^{[k-1]}$:

$$\mathbf{z}^{[k]} = \mathbf{W}^{[k]}\sigma\left(\mathbf{z}^{[k-1]}\right) + \mathbf{b}^{[k]}.$$

Lấy đạo hàm riêng từng phần tử, với i chỉ số hàng của $\mathbf{z}^{[k]}$ và j chỉ số hàng của $\mathbf{z}^{[k-1]}$:

$$\frac{\partial z_i^{[k]}}{\partial z_j^{[k-1]}} = W_{ij}^{[k]}\,\sigma'\left(z_j^{[k-1]}\right).$$

Gom lại thành ma trận, ta được Jacobian tiến

$$\frac{\partial \mathbf{z}^{[k]}}{\partial \mathbf{z}^{[k-1]}} = \mathbf{W}^{[k]}\,\mathrm{diag}\left(\sigma'\left(\mathbf{z}^{[k-1]}\right)\right) \in \mathbb{R}^{n_k \times n_{k-1}}. \tag{3}$$

Ma trận đường chéo nằm *bên phải* vì σ' mang chỉ số j của tầng nguồn.

II.3. Công thức đệ quy của tín hiệu lỗi

Định lý — công thức đệ quy của tín hiệu lỗi

Với quy ước ở Mục II.1,

$$\boldsymbol{\delta}^{[l]} = \mathrm{diag}\left(\sigma'\left(\mathbf{z}^{[l]}\right)\right)\left(\mathbf{W}^{[l+1]}\right)^{\top}\boldsymbol{\delta}^{[l+1]}. \tag{4}$$

Chứng minh. Áp dụng quy tắc dây chuyền cho từng thành phần. Mất mát $\mathcal{L}$ phụ thuộc $\mathbf{z}^{[l]}$ chỉ thông qua $\mathbf{z}^{[l+1]}$, nên

$$\delta_i^{[l]} = \frac{\partial \mathcal{L}}{\partial z_i^{[l]}} = \sum_m \frac{\partial \mathcal{L}}{\partial z_m^{[l+1]}}\frac{\partial z_m^{[l+1]}}{\partial z_i^{[l]}} = \sum_m \delta_m^{[l+1]}\,W_{mi}^{[l+1]}\,\sigma'\left(z_i^{[l]}\right),$$

trong đó bước cuối dùng (3) với $k = l+1$. Nhân tử $\sigma'(z_i^{[l]})$ không phụ thuộc m nên đưa được ra ngoài tổng:

$$\delta_i^{[l]} = \sigma'\left(z_i^{[l]}\right)\sum_m W_{mi}^{[l+1]}\,\delta_m^{[l+1]} = \sigma'\left(z_i^{[l]}\right)\left[\left(\mathbf{W}^{[l+1]}\right)^{\top}\boldsymbol{\delta}^{[l+1]}\right]_i.$$

Tổng $\sum_m W_{mi}^{[l+1]}\delta_m^{[l+1]}$ chạy theo chỉ số *hàng* m của $\mathbf{W}^{[l+1]}$, tức là tích của *chuyển vị* với $\boldsymbol{\delta}^{[l+1]}$ — đây chính là nơi dấu chuyển vị sinh ra. Viết phép nhân theo từng phần tử với σ' dưới dạng nhân ma trận đường chéo bên trái, ta thu được (4). □

Từ $\boldsymbol{\delta}^{[l]}$, gradient theo trọng số có dạng tích ngoài:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{[l]}} = \boldsymbol{\delta}^{[l]}\left(\mathbf{a}^{[l-1]}\right)^{\top}, \qquad \frac{\partial \mathcal{L}}{\partial \mathbf{b}^{[l]}} = \boldsymbol{\delta}^{[l]}. \tag{5}$$

Kiểm tra chiều — cách tự soát lỗi chuyển vị

Một công thức lan truyền ngược sai dấu chuyển vị hầu như luôn sai cả về chiều. Vì vậy phép kiểm tra chiều là công cụ soát lỗi rẻ nhất. Với (4):

$$\underbrace{\operatorname{diag}\big(\sigma'(\mathbf{z}^{[l]})\big)}_{n_l\times n_l}\cdot\underbrace{\big(\mathbf{W}^{[l+1]}\big)^{\top}}_{n_l\times n_{l+1}}\cdot\underbrace{\boldsymbol{\delta}^{[l+1]}}_{n_{l+1}\times 1}=\underbrace{\boldsymbol{\delta}^{[l]}}_{n_l\times 1}\quad\checkmark$$

Nếu viết nhầm thành $\mathbf{W}^{[l+1]}\boldsymbol{\delta}^{[l+1]}$ (không chuyển vị), tích $n_{l+1}\times n_l$ nhân $n_{l+1}\times 1$ không hợp lệ trừ khi $n_l=n_{l+1}$ — đó là lý do lỗi này thường ẩn mình trong các mạng có mọi tầng cùng độ rộng và chỉ lộ ra khi đổi kiến trúc. Tương tự với (5): $(n_l\times 1)(1\times n_{l-1})=n_l\times n_{l-1}$, đúng bằng chiều của $\mathbf{W}^{[l]}$.

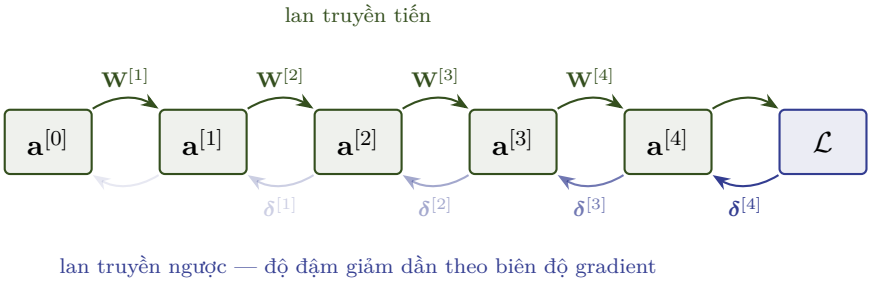
II.4. Đối tượng trung tâm: tích Jacobian

Khai triển (4) đệ quy từ tầng L về tầng l cho kết quả then chốt của toàn bộ tài liệu:

$$\boxed{\boldsymbol{\delta}^{[l]}=\left(\prod_{k=l+1}^{L}\operatorname{diag}\big(\sigma'\big(\mathbf{z}^{[k-1]}\big)\big)\big(\mathbf{W}^{[k]}\big)^{\top}\right)\boldsymbol{\delta}^{[L]}}\tag{6}$$

với quy ước các thừa số xếp theo k tăng dần từ trái sang phải.

Kết hợp (6) với (5), ta thấy gradient của tầng l là tích của $L-l$ thừa số ma trận. Đây là một *tích*, không phải một tổng. Tổng suy giảm tuyến tính; tích suy giảm theo cấp số nhân. Toàn bộ hiện tượng triệt tiêu và bùng nổ gradient là hệ quả trực tiếp của sự khác biệt đó, và Mục III. chỉ làm một việc: ước lượng độ lớn của mỗi thừa số trong (6). Hình 1 vẽ lại cấu trúc này trên đồ thị tính toán: tín hiệu lỗi đi ngược qua từng tầng và gặp một thừa số mới ở mỗi bước.



Hình 1: Đồ thị tính toán của mạng 4 tầng. Mũi tên phía trên là lan truyền tiến; mũi tên phía dưới là lan truyền ngược, với độ đậm giảm dần về phía đầu vào để biểu thị sự suy giảm biên độ của $\boldsymbol{\delta}^{[l]}$ theo (6).

III. Hai nguyên nhân cốt lõi

Phương trình (6) biểu diễn gradient của tầng l như một tích gồm $L-l$ thừa số, mỗi thừa số có dạng

$$\mathbf{M}^{[k]} = \operatorname{diag}\Big(\sigma'\big(\mathbf{z}^{[k-1]}\big)\Big)\,\big(\mathbf{W}^{[k]}\big)^{\top}.$$

Mỗi thừa số là tích của *hai* đối tượng, và mỗi đối tượng đóng góp một nguyên nhân độc lập: ma trận đường chéo mang thông tin về hàm kích hoạt, ma trận chuyển vị mang thông tin về trọng số. Ta phân tích lần lượt rồi hợp nhất.

III.1. Nguyên nhân 1 — độ bão hoà của hàm kích hoạt

Chặn trên của đạo hàm sigmoid

Với $\sigma(z) = (1+e^{-z})^{-1}$, ta có

$$\sigma'(z) = \sigma(z)\big(1-\sigma(z)\big) \qquad \text{và} \qquad \max_{z\in\mathbb{R}} \sigma'(z) = \frac{1}{4},$$

đạt được *duy nhất* tại $z=0$.

Chứng minh. Đạo hàm trực tiếp cho $\sigma'(z) = e^{-z}\,(1+e^{-z})^{-2}$. Viết lại theo σ:

$$\sigma'(z) = \frac{1}{1+e^{-z}} \cdot \frac{e^{-z}}{1+e^{-z}} = \sigma(z)\big(1-\sigma(z)\big).$$

Đặt $u = \sigma(z) \in (0,1)$. Hàm $u \mapsto u(1-u)$ là parabol lõm xuống, đạt cực đại tại $u = 1/2$ với giá trị $1/4$. Vì σ là song ánh tăng nghiêm ngặt từ $\mathbb{R}$ lên $(0,1)$, giá trị $u = 1/2$ ứng với đúng một điểm $z = 0$. Do đó $\sigma'(z) \le 1/4$ với mọi z, dấu bằng chỉ tại gốc. □

Hệ quả trực tiếp: ma trận đường chéo trong mỗi thừa số có chuẩn phổ $\|\operatorname{diag}(\sigma'(\mathbf{z}))\|_2 = \max_i \sigma'(z_i) \le 1/4$. Ngay cả trong trường hợp *thuận lợi nhất* — mọi tiền kích hoạt đều đúng bằng 0 — mỗi tầng vẫn nhân gradient với một hệ số không quá $1/4$. Nếu $\big\|\mathbf{W}^{[k]}\big\|_2 \approx 1$, biên độ gradient co lại theo 4^{-L}.

Bảng 1 cho thấy tốc độ của hiện tượng:

Bảng 1: Đại lượng 0.25^L theo số tầng L. Với $L = 10$ giá trị chỉ còn khoảng 8 lần epsilon máy `float32`; với $L = 20$ nó đã dưới epsilon máy.

Số tầng L	0.25^L	Diễn giải
1	2.500000×10^{-1}	suy giảm nhẹ, chưa đáng ngại
5	9.765625×10^{-4}	tầng đầu học chậm hơn tầng cuối $\sim 10^3$ lần
10	9.536743×10^{-7}	chỉ còn 8 lần epsilon máy `float32`
20	9.094947×10^{-13}	dưới ngưỡng phân giải của `float32`
50	7.888609×10^{-31}	bằng không về mặt số học

Con số ở dòng $L = 10$ đáng được nhấn mạnh. Epsilon máy của `float32` là $\varepsilon_{32} \approx 1.192093 \times 10^{-7}$. Giá trị $0.25^{10} \approx 9.536743 \times 10^{-7}$ chỉ lớn hơn ε_{32} đúng 8 lần. Nghĩa là với một mạng sigmoid 10 tầng huấn luyện ở độ chính xác đơn, gradient tầng đầu đã tiệm cận giới hạn biểu diễn của kiểu số: phép cộng $w \leftarrow w - \eta g$ với g nhỏ như vậy bị làm tròn về chính w, và tầng đó *đóng băng hoàn toàn*. Ở $L = 20$, giá trị 9.094947×10^{-13} đã nằm sâu dưới ε_{32}.

III.1.1. Hàm tanh làm chậm vấn đề nhưng không loại bỏ nó

Với tanh, ta có $\tanh'(z) = 1 - \tanh^2(z)$, và

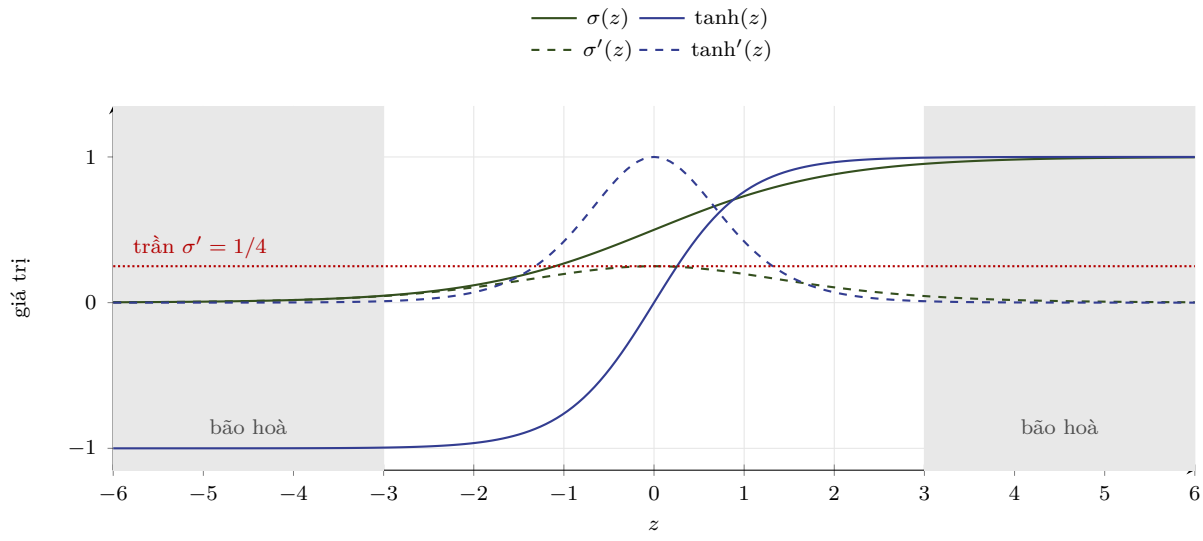
$$\max_{z \in \mathbb{R}} \tanh'(z) = 1,$$

đạt được duy nhất tại $z = 0$. Chặn trên bằng 1 thay vì 1/4 là một cải thiện thực chất: tại chế độ tuyến tính quanh gốc, tanh bảo toàn biên độ gradient trong khi sigmoid làm co lại bốn lần mỗi tầng.

Hình 2 vẽ σ, tanh và đạo hàm của chúng, với vùng bão hoà tô xám. Tuy nhiên cải thiện này chỉ tồn tại *tại một điểm duy nhất*. Với $|z| \geq 2$ đã có $\tanh'(z) \leq 0.072$, nhỏ hơn cả chặn 1/4 của sigmoid. Trong một mạng thật, các tiền kích hoạt phân tán trên một khoảng chứ không tập trung tại 0, nên giá trị $\tanh'$ *trung bình* luôn nhỏ hơn 1 ngặt. Tích của L số nhỏ hơn 1 ngặt vẫn hội tụ về 0 theo cấp số nhân — chỉ với cơ số lớn hơn.

Lưu ý

tanh dời ngưỡng chiều sâu mà vấn đề trở nên nghiêm trọng, chứ không thay đổi bản chất cấp số nhân của nó. Điều này gợi ý vì sao việc chuyển từ sigmoid sang tanh có thể giúp huấn luyện mạng 10 tầng nhưng vẫn thất bại ở mạng 50 tầng; tài liệu không thí nghiệm với tanh.



Hình 2: Sigmoid và tanh cùng đạo hàm của chúng trên $[-6, 6]$. Vùng xám là miền bão hoà, nơi đạo hàm tiến về 0. Đường chấm đỏ là trần $\sigma' = 1/4$ mà đạo hàm sigmoid không bao giờ vượt qua.

III.2. Nguyên nhân 2 — bán kính phổ của ma trận trọng số

Nguyên nhân thứ hai độc lập hoàn toàn với hàm kích hoạt: ngay cả với một mạng tuyến tính ($\sigma = \mathrm{id}$, mọi $\sigma' = 1$), tích các ma trận trọng số vẫn có thể suy biến.

Ký hiệu $\sigma_{\max}(\mathbf{A}) = \|\mathbf{A}\|_2$ là giá trị kỳ dị lớn nhất (chuẩn phổ). Tính chất dưới nhân của chuẩn phổ, $\|\mathbf{AB}\|_2 \le \|\mathbf{A}\|_2 \|\mathbf{B}\|_2$, cho ngay

Chặn trên của tích ma trận

$$\left\| \left(\prod_k \mathbf{W}^{[k]} \right) \mathbf{v} \right\| \;\le\; \left(\prod_k \sigma_{\max}\left(\mathbf{W}^{[k]}\right) \right) \|\mathbf{v}\|.$$

Hệ quả là một phép phân loại sạch sẽ theo ba chế độ:

- Nếu $\sigma_{\max}\left(\mathbf{W}^{[k]}\right) < 1$ với mọi k, tích các chặn là một cấp số nhân công bội nhỏ hơn 1: gradient **suy giảm theo cấp số nhân**.
- Nếu $\sigma_{\max}\left(\mathbf{W}^{[k]}\right) > 1$ với mọi k, chặn trên phân kỳ. Trường hợp này không tự động kéo theo bùng nổ (chặn trên lớn không bắt buộc tích phải lớn), nhưng nếu thêm giả thiết các hướng kỳ dị trội không trực giao nhau — điều xảy ra với xác suất cao khi trọng số khởi tạo ngẫu nhiên — thì biên độ thực sự **tăng theo cấp số nhân**.
- Chế độ biên $\sigma_{\max} = 1$ là không bền: nhiễu số học bất kỳ cũng đẩy hệ sang một trong hai phía.

Đây chính là lý do *khởi tạo* trọng số là một quyết định thiết kế chứ không phải chi tiết kỹ thuật. Xavier [4] và He [5] không làm gì khác ngoài việc chọn phương sai khởi tạo sao cho $\sigma_{\max}(\mathbf{W})$ nằm gần 1 ngay từ bước đầu tiên.

III.3. Hợp nhất: Chặn trên bảo thủ của hệ số co giãn Jacobian

Ghép hai nguyên nhân lại. Áp dụng tính dưới nhân của chuẩn phổ, $\|\mathbf{AB}\|_2 \le \|\mathbf{A}\|_2 \|\mathbf{B}\|_2$, cho từng thừa số của (6):

$$\left\|\mathbf{M}^{[k]}\right\|_2 \;\le\; \underbrace{\left\|\mathrm{diag}\left(\sigma'(\mathbf{z}^{[k-1]})\right)\right\|_2}_{\le \max \sigma'} \cdot \underbrace{\left\|\left(\mathbf{W}^{[k]}\right)^\top\right\|_2}_{= \sigma_{\max}(\mathbf{W}^{[k]})}.$$

Do đó

$$\left\|\boldsymbol{\delta}^{[l]}\right\| \;\le\; c^{L-l} \left\|\boldsymbol{\delta}^{[L]}\right\|, \qquad \boxed{c \;=\; \sigma_{\max}(\mathbf{W}) \cdot \max \sigma'} \tag{7}$$

với c là *chặn trên* của hệ số co giãn mỗi tầng, không phải giá trị thực của nó.

Vì sao chặn này bảo thủ. Bất đẳng thức (7) thay mỗi thừa số bằng trường hợp xấu nhất của nó. Hệ số co giãn thực tế của một tầng, $\left\|\mathbf{M}^{[k]}\mathbf{v}\right\| / \|\mathbf{v}\|$ với $\mathbf{v}$ là vector sai số đang lan truyền ngược, còn phụ thuộc vào hai yếu tố mà c bỏ qua:

(a) **Dữ liệu, qua tiền kích hoạt.** Ma trận $\mathrm{diag}(\sigma'(\mathbf{z}^{[k-1]}))$ được tính tại tiền kích hoạt thực $\mathbf{z}^{[k-1]}$, vốn phụ thuộc đầu vào $\mathbf{x}$ và mọi tầng phía trước. Ở vùng bão hoà của sigmoid, hoặc tại các nơ-ron ReLU chết ($\sigma' = 0$), phần tử đường chéo nhỏ hơn hẳn $\max \sigma'$.

(b) **Sự thẳng hàng với các hướng kỳ dị.** Đẳng thức $\left\|(\mathbf{W}^{[k]})^\top \mathbf{v}\right\| = \sigma_{\max}(\mathbf{W}^{[k]})\,\|\mathbf{v}\|$ chỉ xảy ra khi $\mathbf{v}$ nằm dọc vector kỳ dị trội. Một vector lan truyền ngược bất kỳ chỉ đặt một phần năng lượng lên các hướng đó, nên hệ số co giãn thực nằm đâu đó giữa $\sigma_{\min}$ và $\sigma_{\max}$.

Vì vậy các kết luận rút ra được từ c là kết luận *một chiều*:

- $c < 1$ là *điều kiện đủ* để gradient co lại theo cấp số nhân: chặn trên $c^{L-l}\left\|\boldsymbol{\delta}^{[L]}\right\|$ đã giảm theo hàm mũ thì $\left\|\boldsymbol{\delta}^{[l]}\right\|$, bị chặn bởi nó, cũng phải giảm ít nhất nhanh như vậy.
- $c > 1$ chỉ làm cho bùng nổ *có thể xảy ra*, không bảo đảm nó xảy ra: chặn trên tăng không kéo theo giá trị thực tăng, vì (a) và (b) có thể giữ hệ số co giãn thực dưới 1.
- $c \approx 1$ chỉ bảo đảm chặn trên không co hay giãn theo hàm mũ; giá trị thực vẫn có thể suy giảm vì (a) và (b). Mục VI. đo trực tiếp khoảng cách giữa chặn và giá trị thực.

Bây giờ thay số cho sigmoid. Vì $\max \sigma' = 1/4$, để chặn trên đạt $c \geq 1$ ta cần

$$\sigma_{\max}(\mathbf{W}) \;\geq\; \frac{1}{\max \sigma'} \;=\; 4.$$

Nghĩa là ma trận trọng số phải có chuẩn phổ vượt 4 *chỉ để hoà vốn*. Nhưng một ma trận với $\sigma_{\max} = 4$ khuếch đại tín hiệu *tiến* lên bốn lần mỗi tầng, đẩy các tiền kích hoạt $\mathbf{z}$ ra xa gốc — đúng vào vùng bão hoà, nơi σ' sụp đổ về 0 và giả thiết $\max \sigma' = 1/4$ không còn đúng. Hệ thống tự phá huỷ điều kiện hoà vốn của chính nó.

III.4. Nghịch lý phương sai: vì sao không thể phóng đại trọng số

Lập luận ở cuối mục trước mới chỉ là một khẳng định định tính. Mục này chứng minh nó: điều kiện $c \geq 1$ *không thể* đạt được bằng cách tăng thang đo trọng số, vì chính việc tăng đó phá huỷ giả thiết mà nó dựa vào.

III.4.1. Truy hồi phương sai theo chiều tiến

Giả sử tại thời điểm khởi tạo:

(G1) $W_{ij}^{[l]} \sim \mathcal{N}(0, s^2)$ độc lập cùng phân phối, mọi tầng rộng n;

(G2) các trọng số độc lập với các kích hoạt đầu vào của cùng tầng;

(G3) các thành phần $a_j^{[l-1]}$ không tương quan với nhau;

(G4) hàm kích hoạt hoạt động ở chế độ gần tuyến tính quanh gốc, tức $\sigma(z) \approx \sigma(0) + \sigma'(0)z$ với $\sigma'(0)$ là một hằng số bậc đơn vị.

Đây là các giả thiết *bậc một*, không phải đồng nhất thức. Chúng đúng tại bước khởi tạo ngẫu nhiên và xấu đi khi huấn luyện tiến triển — một điểm sẽ quay lại ở Mục VII. khi bàn về chuẩn hoá.

Với $z_i^{[l]} = \sum_{j=1}^{n} W_{ij}^{[l]} a_j^{[l-1]}$, do (G1)–(G3) và $\mathbb{E}[W] = 0$:

$$\mathrm{Var}\left(z_i^{[l]}\right) = \sum_{j=1}^{n} \mathrm{Var}\left(W_{ij}^{[l]}\right) \mathrm{Var}\left(a_j^{[l-1]}\right) = n\, s^2\, \mathrm{Var}\left(a^{[l-1]}\right).$$

Dùng (G4) để thay $\mathrm{Var}(a^{[l-1]}) \approx \sigma'(0)^2\,\mathrm{Var}(z^{[l-1]})$ và gộp hằng số $\sigma'(0)^2$ vào s^2, ta được truy hồi

$$\mathrm{Var}\big(\mathbf{z}^{[l]}\big) \;\approx\; \big(ns^2\big)\,\mathrm{Var}\big(\mathbf{z}^{[l-1]}\big) \;\Longrightarrow\; \mathrm{Var}\big(\mathbf{z}^{[l]}\big) \;\approx\; \big(ns^2\big)^{l}\,\mathrm{Var}(\mathbf{x})\,. \tag{8}$$

Đại lượng ns^2 đóng vai trò hệ số khuếch đại phương sai mỗi tầng, hoàn toàn song song với c ở (7) nhưng theo chiều ngược lại.

III.4.2. Thế kẹt

Với ma trận ngẫu nhiên $n \times n$ có phương sai phần tử s^2, định luật Bai–Yin cho $\sigma_{\max}(\mathbf{W}) \approx 2s\sqrt{n}$. Điều kiện hoà vốn $\sigma_{\max}(\mathbf{W}) \geq 4$ của mục trước do đó tương đương

$$2s\sqrt{n} \;\geq\; 4 \quad\Longleftrightarrow\quad ns^2 \;\geq\; 4.$$

Thay vào (8):

$$\mathrm{Var}\big(\mathbf{z}^{[l]}\big) \;\gtrsim\; 4^{l}\,\mathrm{Var}(\mathbf{x}), \qquad \text{tức} \qquad \big|z^{[l]}\big| \;\sim\; 2^{l}\;|x|\,.$$

Độ lớn tiền kích hoạt tăng theo cấp số nhân với chiều sâu. Nhưng ở vùng $|z|$ lớn, $\sigma(z) \to 1$ và $1 - \sigma(z) \approx e^{-|z|}$, nên

$$\sigma'(z) = \sigma(z)\big(1-\sigma(z)\big) \;\approx\; e^{-|z|} \qquad (|z| \gg 1). \tag{9}$$

Nghĩa là σ' suy giảm **theo hàm mũ** của $|z|$, trong khi $|z|$ lại tăng theo *cấp số nhân* của l. Ghép hai điều đó:

$$\max \sigma'\Big|_{\text{tầng } l} \;\sim\; e^{-2^{l}|x|},$$

một sự sụp đổ nhanh hơn mọi thứ mà việc nâng $\sigma_{\max}(\mathbf{W})$ từ 1 lên 4 có thể bù lại — vế sau chỉ là một thừa số hằng bằng 4, vế trước là hàm mũ kép theo chiều sâu.

Lưu ý

Bùng nổ phương sai tiến ĐỔI LẤY triệt tiêu gradient ngược. Với hàm kích hoạt bão hoà, $c = \sigma_{\max}(\mathbf{W}) \cdot \max \sigma'$ *không thể* được đưa về 1 bằng cách chỉnh thang đo trọng số: mọi mức tăng s đủ lớn để nâng $\sigma_{\max}$ đều đẩy $\mathbf{z}$ ra vùng bão hoà theo (8), nơi $\max \sigma'$ sụp đổ theo hàm mũ (9). Hai thừa số của c không độc lập — chúng bị ràng buộc ngược chiều qua chính thang đo trọng số.

Kết luận này có giá trị kiến tạo: nó *loại trừ* toàn bộ một họ giải pháp và chỉ ra ba lối thoát chính, mỗi lối phá vỡ một mắt xích của thế kẹt:

1. **Đổi hàm kích hoạt** — nâng $\max \sigma'$ lên 1 mà không đụng tới s, phá vỡ (9). Đây là ReLU và các biến thể [5].
2. **Tái định tâm z** — giữ $\mathrm{Var}(\mathbf{z}^{[l]})$ không đổi theo l bất chấp (8), tức cưỡng bức $ns^2 \approx 1$ ở mọi tầng và mọi bước huấn luyện. Đây là ý tưởng chung của khởi tạo Xavier [4] (đúng tại bước 0) và của chuẩn hoá [6] (duy trì suốt quá trình).
3. **Đổi cấu trúc Jacobian** — thêm số hạng bậc không để tích thôi là tích thuần tuý, khiến câu hỏi về c trở nên không còn quyết định. Đây là kết nối tắt [7].

Ba lối thoát chính này là ba cột đầu của bảng tổng kết ở Mục VII.; chúng không phải một danh sách mẹo rời rạc mà là các cách phá vỡ thế kẹt suy ra từ (7) và (8). Danh sách này không vét cạn: khởi tạo trực giao theo hướng Dynamical Isometry là một đường tiếp cận nữa, xem Mục III.5. ngay sau đây.

Lưu ý

Triệt tiêu và bùng nổ không phải hai lỗi khác nhau. Chúng là cùng một cơ chế — tích lặp của các Jacobian tầng — quan sát ở hai phía của ngưỡng $c = 1$. Mọi biện pháp khắc phục ở Mục VII. đều là hệ quả của (7): hoặc nâng $\max \sigma'$, hoặc điều khiển $\sigma_{\max}(\mathbf{W})$, hoặc giữ $\mathbf{z}$ ngoài vùng bão hoà, hoặc thay đổi cấu trúc Jacobian để nó không còn là tích thuần tuý.

III.5. Dynamical Isometry: phổ của toàn bộ Jacobian

Chặn c ở (7) chỉ nhìn vào độ lớn *tối đa* của từng thừa số. Để biết *mọi hướng* của tín hiệu lỗi có được truyền về hay không, ta cần phổ của cả tích Jacobian, không chỉ chuẩn phổ của nó.

Định nghĩa. Giả sử các tầng ẩn $1, \dots, L-1$ cùng rộng n và $L \geq 3$. Theo (3), Jacobian đầu-cuối của chồng tầng ẩn là

$$\mathbf{J} := \frac{\partial \mathbf{z}^{[L-1]}}{\partial \mathbf{z}^{[1]}} = \mathbf{W}^{[L-1]}\mathbf{D}^{[L-2]} \cdots \mathbf{W}^{[2]}\mathbf{D}^{[1]} \in \mathbb{R}^{n\times n}, \qquad \mathbf{D}^{[k]} := \operatorname{diag}\left(\sigma'\left(\mathbf{z}^{[k]}\right)\right), \tag{10}$$

gồm $L-2$ thừa số $\mathbf{W}^{[k]}\mathbf{D}^{[k-1]}$. $\mathbf{J}$ cố ý *loại tầng ra*: nếu tầng ra có một nơ-ron (như ở Mục VI.), Jacobian tới đầu ra là một ma trận $1 \times n$ có hạng không quá 1, phổ của nó suy biến và không mang thông tin.

Mệnh đề — tín hiệu lỗi đi qua Jacobian đầu-cuối

Với $\mathbf{J}$ như ở (10) và mọi vector $\boldsymbol{\delta}^{[L-1]} \in \mathbb{R}^n$,

$$\boldsymbol{\delta}^{[1]} = \mathbf{D}^{[1]}\left(\mathbf{W}^{[2]}\right)^\top \mathbf{D}^{[2]}\left(\mathbf{W}^{[3]}\right)^\top \cdots \mathbf{D}^{[L-2]}\left(\mathbf{W}^{[L-1]}\right)^\top \boldsymbol{\delta}^{[L-1]} = \mathbf{J}^\top \boldsymbol{\delta}^{[L-1]}, \tag{11}$$

và

$$\sigma_{\min}(\mathbf{J})\left\|\boldsymbol{\delta}^{[L-1]}\right\| \leq \left\|\boldsymbol{\delta}^{[1]}\right\| \leq \sigma_{\max}(\mathbf{J})\left\|\boldsymbol{\delta}^{[L-1]}\right\|, \tag{12}$$

trong đó cả hai dấu bằng đều đạt được với $\boldsymbol{\delta}^{[L-1]}$ chọn thích hợp.

Chứng minh. Bước 1 (quy nạp theo tầng). Với $1 \leq l \leq L-2$ đặt

$$\mathbf{P}_l := \mathbf{D}^{[l]}\left(\mathbf{W}^{[l+1]}\right)^\top \mathbf{D}^{[l+1]} \cdots \mathbf{D}^{[L-2]}\left(\mathbf{W}^{[L-1]}\right)^\top.$$

Ta chứng minh $\boldsymbol{\delta}^{[l]} = \mathbf{P}_l \boldsymbol{\delta}^{[L-1]}$. Với $l = L-2$, (4) cho $\boldsymbol{\delta}^{[L-2]} = \mathbf{D}^{[L-2]}(\mathbf{W}^{[L-1]})^\top \boldsymbol{\delta}^{[L-1]} = \mathbf{P}_{L-2}\boldsymbol{\delta}^{[L-1]}$. Nếu đúng với l thì áp dụng (4) cho tầng $l-1$:

$$\boldsymbol{\delta}^{[l-1]} = \mathbf{D}^{[l-1]}\left(\mathbf{W}^{[l]}\right)^\top \boldsymbol{\delta}^{[l]} = \mathbf{D}^{[l-1]}\left(\mathbf{W}^{[l]}\right)^\top \mathbf{P}_l \boldsymbol{\delta}^{[L-1]} = \mathbf{P}_{l-1}\boldsymbol{\delta}^{[L-1]}.$$

Sau $L-2$ lần áp dụng ta được $\boldsymbol{\delta}^{[1]} = \mathbf{P}_1 \boldsymbol{\delta}^{[L-1]}$.

Bước 2 (nhận ra $\mathbf{J}^\top$). $\mathbf{D}^{[k]}$ đối xứng và chuyển vị đảo thứ tự của một tích, nên

$$\mathbf{J}^\top = \left(\mathbf{W}^{[L-1]}\mathbf{D}^{[L-2]}\cdots\mathbf{W}^{[2]}\mathbf{D}^{[1]}\right)^\top = \mathbf{D}^{[1]}\left(\mathbf{W}^{[2]}\right)^\top\cdots\mathbf{D}^{[L-2]}\left(\mathbf{W}^{[L-1]}\right)^\top = \mathbf{P}_1,$$

suy ra (11).

Bước 3 (bất đẳng thức SVD). Viết $\mathbf{J} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top$ với $\mathbf{U},\mathbf{V}$ trực giao $n\times n$ và $\mathbf{\Sigma} = \mathrm{diag}(\sigma_1 \geq \cdots \geq \sigma_n \geq 0)$. Khi đó $\mathbf{J}^\top\boldsymbol{\delta} = \mathbf{V}\mathbf{\Sigma}\mathbf{U}^\top\boldsymbol{\delta}$ và, vì $\mathbf{V}$ trực giao,

$$\left\|\mathbf{J}^\top\boldsymbol{\delta}\right\|^2 = \sum_{i=1}^{n}\sigma_i^2\left(\mathbf{u}_i^\top\boldsymbol{\delta}\right)^2, \qquad \sum_{i=1}^{n}\left(\mathbf{u}_i^\top\boldsymbol{\delta}\right)^2 = \|\boldsymbol{\delta}\|^2$$

(đẳng thức sau do $\mathbf{U}$ trực giao). Tổng thứ nhất là trung bình có trọng số của σ_i^2 nhân $\|\boldsymbol{\delta}\|^2$, nên nằm giữa $\sigma_n^2\|\boldsymbol{\delta}\|^2$ và $\sigma_1^2\|\boldsymbol{\delta}\|^2$; lấy căn bậc hai được (12). Dấu bằng xảy ra khi $\boldsymbol{\delta} = \mathbf{u}_n$ (vế trái) hoặc $\mathbf{u}_1$ (vế phải). □

Hệ quả. Tính dưới nhân của chuẩn phổ cho $\sigma_{\max}(\mathbf{J}) \leq \prod_{k=2}^{L-1}\left\|\mathbf{W}^{[k]}\right\|_2 \max_i\left|\sigma'(z_i^{[k-1]})\right|$, tức chặn kiểu c^{L-2} của (7). Điểm khác biệt là (12) là chặn *hai chiều* và đúng cho *mọi* vector $\boldsymbol{\delta}^{[L-1]}$, trong khi c chỉ cho vế phải. Vế trái cho thấy điều mà c không nói được: nếu $\sigma_{\min}(\mathbf{J})$ rất nhỏ thì tồn tại những hướng mà tín hiệu lỗi hầu như không về tới tầng 1, dù $\sigma_{\max}(\mathbf{J})$ lớn. Tỉ số $\kappa(\mathbf{J}) = \sigma_{\max}/\sigma_{\min}$ đo mức độ bất đồng đều đó. (Gradient theo $\mathbf{W}^{[1]}$ còn nhân thêm $(\mathbf{a}^{[0]})^\top$ theo (5), nên so sánh với chuẩn gradient ở Mục VI. chỉ có ý nghĩa ở bậc độ lớn.)

Dynamical Isometry. Xiao và cộng sự [8] gọi *dynamical isometry* là sự cân bằng (equilibration) các giá trị kỳ dị của Jacobian đầu vào–đầu ra. Trong tài liệu này ta dùng dạng cụ thể nhất của nó, là cách diễn đạt của tài liệu chứ không trích từ tóm tắt bài báo: mọi $\sigma_i(\mathbf{J}) \approx 1$, khi đó $\|\boldsymbol{\delta}^{[1]}\| \approx \|\boldsymbol{\delta}^{[L-1]}\|$ cho *mọi* vector theo (12), không chỉ trung bình. Khởi tạo Xavier và He nhắm vào một điều kiện yếu hơn. Nếu $\boldsymbol{\delta}^{[L-1]}$ có phân phối đẳng hướng (cùng phương sai s^2 trên mọi toạ độ, không tương quan) thì $\mathbb{E}\|\mathbf{J}^\top\boldsymbol{\delta}^{[L-1]}\|^2 = s^2\|\mathbf{J}\|_F^2 = ns^2\cdot\mathrm{RMS}(\sigma)^2$, với $\mathrm{RMS}(\sigma) = \sqrt{n^{-1}\sum_i\sigma_i^2}$. Vì vậy việc bảo toàn *phương sai* tín hiệu, theo (G1)–(G3), tương ứng với $\mathrm{RMS}(\sigma) \approx 1$: một đại lượng trung bình bậc hai, không ràng buộc từng σ_i. Đây chỉ là một gợi ý; Mục VI.4. đo cả hai đại lượng: ở ReLU + He, $\mathrm{RMS}(\sigma)$ gần 1 trong khi phổ còn rất xa dạng đồng đều (Bảng 5), phù hợp với nhận xét trên; ở sigmoid + Xavier, $\mathrm{RMS}(\sigma)$ đã nhỏ hơn 1 nhiều bậc, tức Xavier không giữ được ngay cả đại lượng trung bình này khi đi qua sigmoid (hệ số $\sigma'(0) = 1/4$ và độ bão hoà không được tính vào s^2).

Khởi tạo trực giao. Xiao và cộng sự [8] cho thấy mạng tích chập thuần, không kết nối tắt và không chuẩn hoá, có thể được huấn luyện với 10,000 tầng trở lên chỉ nhờ một sơ đồ khởi tạo phù hợp; điều kiện dynamical isometry buộc toán tử tích chập là một phép biến đổi trực giao (bảo toàn chuẩn). Đó là đường thứ tư đạt dynamical isometry mà không cần kết nối tắt. Tài liệu này *không* thực nghiệm với khởi tạo trực giao, và chỉ dẫn lại điều được nêu trong tóm tắt của bài báo.

Chuẩn hoá ở mạng thuần. Yang và cộng sự [9] xây dựng lý thuyết trường trung bình cho BatchNorm trong mạng truyền thẳng kết nối đầy đủ, rộng, tại bước khởi tạo: gradient tăng theo hàm mũ của độ sâu, hiện tượng này không loại bỏ được bằng cách chỉnh phương sai khởi tạo

hay đổi hàm kích hoạt, và chính BatchNorm là nguyên nhân. Hệ quả là mạng BatchNorm thuần không có kết nối tắt không huấn luyện được ở độ sâu lớn với các sơ đồ khởi tạo thông dụng. Phần (d) của Mục VI.5. đối chiếu dự báo này với số liệu của PlainNet + BatchNorm.

IV. Minh hoạ số học: chuỗi ba tầng

Mục này kiểm chứng (7) bằng một ví dụ đủ nhỏ để tính tay trọn vẹn. Mọi con số dưới đây được tính ở độ chính xác kép rồi làm tròn tới sáu chữ số thập phân.

Thiết lập bài toán

Mạng vô hướng ba tầng, mỗi tầng một nơ-ron, không có hàm kích hoạt ở đầu ra ngoài sigmoid:

$$z_1 = w_1 x + b, \quad a_1 = \sigma(z_1); \qquad z_2 = w_2 a_1 + b, \quad a_2 = \sigma(z_2); \qquad z_3 = w_3 a_2 + b, \quad a_3 = \sigma(z_3).$$

Tham số và dữ liệu:

$$x = 1.0, \qquad y = 0.0, \qquad w_1 = w_2 = w_3 = 0.5, \qquad b = 0.$$

Hàm mất mát là bình phương sai số **có hệ số** $\frac{1}{2}$:

$$\mathcal{L} = \tfrac{1}{2}\,(a_3 - y)^2 .$$

Hệ số $\frac{1}{2}$ được nêu tường minh vì nó triệt tiêu số 2 khi lấy đạo hàm, cho $\partial \mathcal{L}/\partial a_3 = a_3 - y$. Bỏ qua nó sẽ làm mọi gradient dưới đây gấp đôi.

IV.1. Lan truyền tiến

$$\begin{aligned} z_1 &= 0.5 \times 1.0 + 0 = 0.500000, & a_1 &= \sigma(0.500000) = 0.622459, \\ z_2 &= 0.5 \times 0.622459 = 0.311230, & a_2 &= \sigma(0.311230) = 0.577185, \\ z_3 &= 0.5 \times 0.577185 = 0.288593, & a_3 &= \sigma(0.288593) = 0.571652. \end{aligned}$$

Mất mát:

$$\mathcal{L} = \tfrac{1}{2}\,(0.571652 - 0)^2 = \tfrac{1}{2} \times 0.326786 = 0.163393.$$

IV.2. Lan truyền ngược, từng phép nhân một

Áp dụng (4) ở dạng vô hướng: $\delta_l = \sigma'(z_l)\, w_{l+1}\, \delta_{l+1}$, với $\delta_3 = (a_3 - y)\,\sigma'(z_3)$ và $\partial \mathcal{L}/\partial w_l = \delta_l\, a_{l-1}$ (trong đó $a_0 = x$).

Tầng 3.

$$\begin{aligned} \sigma'(z_3) &= a_3(1 - a_3) = 0.571652 \times 0.428348 = 0.244866, \\ \delta_3 &= (a_3 - y)\,\sigma'(z_3) = 0.571652 \times 0.244866 = 0.139978, \\ \frac{\partial \mathcal{L}}{\partial w_3} &= \delta_3\, a_2 = 0.139978 \times 0.577185 = 0.080793. \end{aligned}$$

Tầng 2.

$$\begin{aligned}\sigma'(z_2) &= a_2(1-a_2) = 0.577185 \times 0.422815 = 0.244042,\\ \delta_2 &= \sigma'(z_2)\, w_3\, \delta_3 = 0.244042 \times 0.5 \times 0.139978 = 0.017080,\\ \frac{\partial \mathcal{L}}{\partial w_2} &= \delta_2\, a_1 = 0.017080 \times 0.622459 = 0.010632.\end{aligned}$$

Tầng 1.

$$\begin{aligned}\sigma'(z_1) &= a_1(1-a_1) = 0.622459 \times 0.377541 = 0.235004,\\ \delta_1 &= \sigma'(z_1)\, w_2\, \delta_2 = 0.235004 \times 0.5 \times 0.017080 = 0.002007,\\ \frac{\partial \mathcal{L}}{\partial w_1} &= \delta_1\, a_0 = 0.002007 \times 1.0 = 0.002007.\end{aligned}$$

IV.3. Đọc kết quả

Mỗi bước lùi một tầng nhân gradient với đúng hai thừa số: $\sigma'(z_l) \le 0.25$ và $w_{l+1} = 0.5$. Hệ số khuếch đại mỗi tầng vì thế là

$$c \;\approx\; 0.244 \times 0.5 \;\approx\; 0.122,$$

phù hợp với (7). Tỉ lệ giữa hai đầu mạng:

$$\frac{\partial \mathcal{L}/\partial w_1}{\partial \mathcal{L}/\partial w_3} = \frac{0.002007}{0.080793} = 0.024841 \;\approx\; \frac{1}{40}.$$

Với cùng một tốc độ học η, tầng 1 nhận bước cập nhật nhỏ hơn tầng 3 khoảng 40 **lần** — và đây mới chỉ là mạng *ba* tầng. Ngoại suy với hệ số $c \approx 0.122$ mỗi tầng, ở $L = 10$ tỉ lệ giữa tầng đầu và tầng cuối là

$$c^9 \approx 0.122^9 \approx 5.99 \times 10^{-9},$$

tức tầng đầu học chậm hơn tầng cuối hơn *một trăm triệu* lần. Ở tốc độ đó, tầng đầu về thực chất giữ nguyên giá trị khởi tạo ngẫu nhiên trong suốt quá trình huấn luyện: mạng 10 tầng hoạt động như một mạng 2–3 tầng đặt sau một phép biến đổi ngẫu nhiên cố định. Đây chính là cơ chế vi mô của *cái bẫy mạng nông* mô tả ở Mục I..

Lưu ý

Ví dụ này dùng $w = 0.5$, tức $\sigma_{\max}(\mathbf{W}) = 0.5 < 1$, nên *cả hai* nguyên nhân ở Mục III. cùng tác động theo một hướng. Ngay cả khi đặt $w = 2$ để bù lại ($c \approx 0.244 \times 2 = 0.488$), gradient vẫn suy giảm — phải tới $w = 4$ mới hoà vốn, và khi đó z bị đẩy vào vùng bão hoà đúng như cảnh báo ở cuối Mục III..

IV.4. Cross-entropy có cứu được không?

Một phản đề thường gặp: "gradient nhỏ là do MSE ghép với sigmoid; dùng cross-entropy là hết". Mục này kiểm tra lập luận đó bằng chính bộ số trên, và cho thấy nó đúng một nửa — đúng ở tầng ra, sai ở các tầng ẩn.

IV.4.1. Triệt tiêu ở tầng ra

Với nhãn nhị phân, mất mát cross-entropy là

$$\mathcal{L}_{\text{BCE}} = -\Big[\, y \log a_3 + (1-y)\log(1-a_3) \,\Big].$$

Đạo hàm theo đầu ra:

$$\frac{\partial \mathcal{L}_{\text{BCE}}}{\partial a_3} = -\frac{y}{a_3} + \frac{1-y}{1-a_3} = \frac{-y(1-a_3) + (1-y)a_3}{a_3(1-a_3)} = \frac{a_3 - y}{a_3\Big(1-a_3\Big)}.$$

Nhân với $\sigma'(z_3) = a_3(1-a_3)$ để được δ_3:

$$\delta_3 = \frac{\partial \mathcal{L}_{\text{BCE}}}{\partial a_3} \cdot \sigma'(z_3) = \frac{a_3 - y}{\underbrace{a_3(1-a_3)}_{\text{mẫu số}}} \cdot \underbrace{a_3(1-a_3)}_{=\sigma'(z_3)} = a_3 - y. \tag{13}$$

Phép triệt tiêu ở (13) **là toàn bộ nội dung của lập luận trên.** Mẫu số của $\partial\mathcal{L}/\partial a_3$ đúng bằng $\sigma'(z_3)$, nên thừa số $\leq 1/4$ biến mất khỏi tầng ra. Với MSE thì không: $\partial\mathcal{L}/\partial a_3 = a_3 - y$ và $\delta_3 = (a_3 - y)\sigma'(z_3)$ vẫn mang thừa số bão hoà.

IV.4.2. Tính lại với cùng bộ số

Giữ nguyên $x = 1.0$, $y = 0$, $w_1 = w_2 = w_3 = 0.5$, $b = 0$, nên lan truyền tiến không đổi: $a_1 = 0.622459$, $a_2 = 0.577185$, $a_3 = 0.571652$. Mất mát:

$$\mathcal{L}_{\text{BCE}} = -\log(1-a_3) = -\log(0.428348) = 0.847818.$$

Tầng 3.

$$\begin{aligned}
\frac{\partial \mathcal{L}}{\partial a_3} &= \frac{0.571652 - 0}{0.571652 \times 0.428348} = \frac{0.571652}{0.244866} = 2.334548, \\
\delta_3 &= 2.334548 \times 0.244866 = 0.571652 \;=\; a_3 - y \quad \checkmark \\
\frac{\partial \mathcal{L}}{\partial w_3} &= \delta_3\, a_2 = 0.571652 \times 0.577185 = 0.329949.
\end{aligned}$$

Tầng 2 và tầng 1. Truy hồi ở các tầng ẩn *không thay đổi* — nó không chứa hàm mất mát:

$$\begin{aligned}
\delta_2 &= \sigma'(z_2)\, w_3\, \delta_3 = 0.244042 \times 0.5 \times 0.571652 = 0.069753, \\
\frac{\partial \mathcal{L}}{\partial w_2} &= \delta_2\, a_1 = 0.069753 \times 0.622459 = 0.043419, \\
\delta_1 &= \sigma'(z_1)\, w_2\, \delta_2 = 0.235004 \times 0.5 \times 0.069753 = 0.008196, \\
\frac{\partial \mathcal{L}}{\partial w_1} &= \delta_1\, a_0 = 0.008196 \times 1.0 = 0.008196.
\end{aligned}$$

IV.4.3. Kết quả quyết định: tỉ lệ suy giảm không phụ thuộc hàm mất mát

Mọi gradient dưới BCE đều lớn hơn dưới MSE — 0.329949 so với 0.080793 ở tầng 3, gấp khoảng 4.08 lần. Nhưng câu hỏi đúng không phải "gradient có lớn hơn không", mà "tầng 1 có bắt kịp tầng 3 không". Ta chứng minh là *không*, và chứng minh ở mức đồng nhất thức chứ không phải xấp xỉ.

Với mọi hàm mất mát khả vi, truy hồi tầng ẩn là $\delta_l = \sigma'(z_l)\, w_{l+1}\, \delta_{l+1}$, nên

$$\delta_1 = \Big[\sigma'(z_1)w_2\Big]\Big[\sigma'(z_2)w_3\Big]\delta_3.$$

Do đó

$$\frac{\partial \mathcal{L}/\partial w_1}{\partial \mathcal{L}/\partial w_3} = \frac{\delta_1\, x}{\delta_3\, a_2} = \frac{\Big[\sigma'(z_1)w_2\Big]\Big[\sigma'(z_2)w_3\Big]\,\delta_3\, x}{\delta_3\, a_2} = \Big[\sigma'(z_1)w_2\Big]\Big[\sigma'(z_2)w_3\Big]\frac{x}{a_2}. \tag{14}$$

δ_3 — đại lượng duy nhất mà hàm mất mát chạm tới — **triệt tiêu hoàn toàn**. Vế phải chỉ còn chứa đạo hàm hàm kích hoạt, trọng số và kích hoạt, không còn dấu vết nào của $\mathcal{L}$. Kiểm chứng bằng số:

$$\Big[0.235004 \times 0.5\Big]\Big[0.244042 \times 0.5\Big]\frac{1.0}{0.577185} = 0.024841,$$

đúng bằng tỉ lệ đã tính ở cả hai hàng của Bảng 2.

Bảng 2: Cross-entropy phóng đại toàn bộ trường gradient (cột 2–4) nhưng không làm phẳng nó: tỉ lệ giữa tầng đầu và tầng cuối ở cột cuối là một con số *giống hệt nhau*, đúng như (14) tiên đoán.

Hàm mất mát	$\boldsymbol{\delta_3}$	$\partial\mathcal{L}/\partial w_3$	$\partial\mathcal{L}/\partial w_1$	**tỉ lệ** $w_1 : w_3$
MSE, $\frac{1}{2}(a_3 - y)^2$	0.139978	0.080793	0.002007	0.024841
Cross-entropy nhị phân	0.571652	0.329949	0.008196	0.024841

Lưu ý

Cross-entropy chữa được hiện tượng "học chậm ở tầng ra khi dự đoán sai nặng" — một vấn đề có thật và khác hẳn. Nó *không* chữa triệt tiêu gradient, vì nguyên nhân của triệt tiêu nằm ở tích $\prod \sigma'(z_k)w_{k+1}$ giữa các tầng ẩn, nơi hàm mất mát không có mặt. Nói ngắn gọn: cross-entropy **đổi thang** trường gradient chứ không **đổi hình dạng** của nó.

V. Lan truyền ngược theo thời gian

Mạng hồi quy (RNN) đưa vấn đề triệt tiêu gradient sang một dạng nghiêm trọng hơn hẳn. Lý do không nằm ở chiều sâu lớn hơn, mà ở một tính chất cấu trúc sẽ được chỉ rõ ở Mục V.3.

Lưu ý

Quy ước ký hiệu cho mục này. T (chữ nghiêng, in thường trong công thức) luôn là *số bước thời gian*. Phép chuyển vị luôn viết bằng $\top$, ví dụ $\mathbf{W}_{hh}^{\top}$. Trong toàn mục này không bao giờ dùng W^T để chỉ chuyển vị — ký hiệu đó sẽ mang nghĩa "luỹ thừa T".

V.1. Mô hình và ký hiệu

Xét RNN với trạng thái ẩn $\mathbf{h}_t \in \mathbb{R}^n$:

$$\mathbf{u}_t = \mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{W}_{xh}\mathbf{x}_t + \mathbf{b}_h, \qquad \mathbf{h}_t = \tanh(\mathbf{u}_t)\,. \tag{15}$$

Ta tách tiền kích hoạt $\mathbf{u}_t$ ra khỏi trạng thái $\mathbf{h}_t$ để giữ đúng quy ước của Mục II., nơi $\boldsymbol{\delta}$ luôn được định nghĩa theo *tiền* kích hoạt. Đặt

$$\boldsymbol{\delta}_t := \frac{\partial \mathcal{L}_T}{\partial \mathbf{u}_t}, \qquad \mathbf{g}_t := \frac{\partial \mathcal{L}_T}{\partial \mathbf{h}_t},$$

với $\mathcal{L}_T$ là mất mát tại bước cuối T. Vì tanh tác động theo từng phần tử và $\tanh' = 1 - \tanh^2$, ta có sẵn $\tanh'(\mathbf{u}_t) = 1 - \mathbf{h}_t^2$ (bình phương theo từng phần tử) — một tiện lợi tính toán: đạo hàm biểu diễn được qua chính đầu ra.

V.2. Công thức đệ quy

Đệ quy BPTT theo tiền kích hoạt

Với quy ước ở Mục II.,

$$\boldsymbol{\delta}_t = \operatorname{diag}\!\left(1 - \mathbf{h}_t^2\right) \mathbf{W}_{hh}^{\top}\, \boldsymbol{\delta}_{t+1}, \tag{16}$$

và do đó

$$\boldsymbol{\delta}_1 = \left(\prod_{t=2}^{T} \operatorname{diag}\!\left(1 - \mathbf{h}_{t-1}^2\right) \mathbf{W}_{hh}^{\top} \right) \boldsymbol{\delta}_T. \tag{17}$$

Chứng minh. Từ (15), $\mathbf{u}_{t+1}$ phụ thuộc $\mathbf{u}_t$ qua $\mathbf{u}_{t+1} = \mathbf{W}_{hh}\tanh(\mathbf{u}_t) + \cdots$, nên

$$\frac{\partial u_{t+1,m}}{\partial u_{t,i}} = W_{hh,\,mi}\left(1 - h_{t,i}^2\right).$$

Quy tắc dây chuyền cho

$$\delta_{t,i} = \sum_m \delta_{t+1,m}\, W_{hh,\,mi}\left(1 - h_{t,i}^2\right) = \left(1 - h_{t,i}^2\right)\left[\mathbf{W}_{hh}^{\top}\boldsymbol{\delta}_{t+1}\right]_i,$$

đúng dạng (16). Lặp lại từ $t = 1$ tới T cho (17). □

Lưu ý

Nếu viết đệ quy theo *trạng thái* $\mathbf{h}$ thay vì tiền kích hoạt, thứ tự hai thừa số đảo lại: $\mathbf{g}_t = \mathbf{W}_{hh}^{\top}\operatorname{diag}(1 - \mathbf{h}_{t+1}^2)\,\mathbf{g}_{t+1}$, vì khi đó phi tuyến nằm ở bước $t+1$ chứ không phải bước t. Hai cách viết hoàn toàn tương đương và cho cùng một chặn; nhầm lẫn giữa chúng là lỗi phổ biến nhất khi cài đặt BPTT bằng tay. Quy ước theo $\mathbf{u}$ ở (16) khớp chính xác với dạng $\operatorname{diag}\cdot\mathbf{W}^{\top}$ của (4).

V.3. Vì sao RNN tệ hơn MLP cùng độ sâu

Áp dụng tính dưới nhân cho (17), giống hệt cách làm ở (7):

$$\|\boldsymbol{\delta}_1\| \;\le\; \left(\sigma_{\max}(\mathbf{W}_{hh}) \cdot \max \tanh'\right)^{T-1} \|\boldsymbol{\delta}_T\| \;\le\; \sigma_{\max}(\mathbf{W}_{hh})^{T-1}\, \|\boldsymbol{\delta}_T\|\,. \tag{18}$$

Điểm cấu trúc then chốt nằm ở chỗ: trong (17), **cùng một ma trận $\mathbf{W}_{hh}$ được nhân lặp lại $T-1$ lần**. Đây là khác biệt cốt yếu so với MLP.

Trong một MLP L tầng, tích (6) gồm L ma trận *khác nhau* $\mathbf{W}^{[1]}, \ldots, \mathbf{W}^{[L]}$. Các giá trị $\sigma_{\max}(\mathbf{W}^{[k]})$ dao động quanh một giá trị trung bình, và tích của chúng chịu một hiệu ứng trung bình hoá: một tầng có $\sigma_{\max} = 0.9$ có thể được bù bởi một tầng có $\sigma_{\max} = 1.1$. Độ lệch tích luỹ tăng theo $\sqrt{L}$ chứ không theo L.

Trong RNN, *không có sự trung bình hoá nào cả*. Chặn trong (18) là $\sigma_{\max}(\mathbf{W}_{hh})^{T-1}$ — một cấp số nhân thuần tuý với công bội cố định. Nếu $\sigma_{\max}(\mathbf{W}_{hh}) = 0.9$, thì sau $T = 100$ bước thời gian hệ số là $0.9^{99} \approx 2.95 \times 10^{-5}$, không có cơ chế nào kéo nó lại. Nói thẳng:

Kết luận so sánh

Một RNN duỗi qua T bước thời gian **tệ hơn ngặt** một MLP T tầng, dù hai mạng có cùng chiều sâu tính toán. Chia sẻ trọng số biến tích các ma trận độc lập thành luỹ thừa của một ma trận duy nhất, loại bỏ hiệu ứng trung bình hoá và biến mọi độ lệch nhỏ của $\sigma_{\max}$ khỏi 1 thành phân kỳ hoặc triệt tiêu theo cấp số nhân [1, 3].

V.4. Phân tích phổ: điều gì thực sự quyết định tốc độ suy giảm

Chặn (18) dùng chuẩn phổ nên là một chặn *trên*. Phân tích riêng cho biết hành vi *thực sự*.

Giả sử $\mathbf{W}_{hh}$ chéo hoá được: $\mathbf{W}_{hh} = \mathbf{Q}\mathbf{\Lambda}\mathbf{Q}^{-1}$ với $\mathbf{\Lambda} = \mathrm{diag}(\lambda_1, \ldots, \lambda_n)$. Khi đó

$$\mathbf{W}_{hh}^{\top} = \left(\mathbf{Q}^{-1}\right)^{\top} \mathbf{\Lambda}\, \mathbf{Q}^{\top}, \qquad \text{nên} \qquad \left(\mathbf{W}_{hh}^{\top}\right)^{T-1} = \left(\mathbf{Q}^{-1}\right)^{\top} \mathbf{\Lambda}^{T-1} \mathbf{Q}^{\top},$$

vì các thừa số $\mathbf{Q}^{\top}(\mathbf{Q}^{-1})^{\top} = \mathbf{I}$ triệt tiêu từng cặp ở giữa. Do $\mathbf{\Lambda}^{T-1} = \mathrm{diag}(\lambda_1^{T-1}, \ldots, \lambda_n^{T-1})$, một thành phần gradient nằm dọc hướng riêng thứ i bị nhân đúng với λ_i^{T-1}:

Suy giảm theo từng hướng riêng

Thành phần gradient dọc hướng riêng ứng với λ_i suy giảm theo cấp số nhân $|\lambda_i|^{T-1}$. Mọi hướng có $|\lambda_i| < 1$ tiêu biến; chỉ các hướng có $|\lambda_i| \geq 1$ còn sống sót qua nhiều bước thời gian. Sau đủ nhiều bước, gradient bị *ép* vào không gian con sinh bởi các hướng riêng trội.

V.4.1. Hai điều kiện cần nói rõ

Thứ nhất, chéo hoá được là một giả thiết. Không phải mọi ma trận đều chéo hoá được; với ma trận khuyết, phải dùng dạng chuẩn Jordan và các khối Jordan sinh thêm thừa số đa thức theo T. Các thừa số đa thức này không đổi bản chất cấp số nhân của kết luận, nhưng làm công thức không còn gọn.

Thứ hai, và quan trọng hơn: với $\mathbf{W}_{hh}$ không chuẩn tắc (*non-normal*, tức $\mathbf{W}_{hh}\mathbf{W}_{hh}^{\top} \neq \mathbf{W}_{hh}^{\top}\mathbf{W}_{hh}$), bán kính phổ chỉ chi phối hành vi *tiệm cận* khi $T \to \infty$. Ở các T hữu hạn, biên độ có thể *tăng vọt tạm thời* vượt xa mức mà $\max_i |\lambda_i|$ gợi ý, do các hướng riêng không trực giao nhau. Một ma trận có mọi $|\lambda_i| < 1$ vẫn có thể khuếch đại gradient hàng chục lần ở những bước trung gian trước khi cuối cùng suy giảm. Đây là lý do $\sigma_{\max}$ — chứ không phải $|\lambda_{\max}|$ — mới là đại lượng dùng để *bảo đảm* an toàn.

Vì vậy phát biểu đúng của Pascanu và cộng sự [1] là bất đối xứng, và cần được trích dẫn đúng như vậy:

Phát biểu chính xác — chú ý tính bất đối xứng

- $\sigma_{\max}(\mathbf{W}_{hh}) < 1$ là điều kiện **đủ** để gradient triệt tiêu. Đây là một bảo đảm: nếu chuẩn phổ nhỏ hơn 1, gradient *chắc chắn* tiêu biến theo cấp số nhân.
- $|\lambda_{\max}(\mathbf{W}_{hh})| > 1$ là điều kiện **cần nhưng không đủ** để gradient bùng nổ. Bán kính phổ lớn hơn 1 mở ra khả năng bùng nổ nhưng không bảo đảm nó xảy ra: hướng riêng trội có thể gần như trực giao với gradient thực tế, hoặc thừa số $\text{diag}(1 - \mathbf{h}_t^2) \leq 1$ có thể ghìm tích lại.

Hệ quả thực hành: dùng $\sigma_{\max}$ để chẩn đoán triệt tiêu, và đừng suy ra "an toàn" chỉ từ việc $|\lambda_{\max}| < 1$.

V.5. Quên thảm hoạ các phụ thuộc xa

Hệ quả thực tiễn của (18) là hiện tượng *quên thảm hoạ* (catastrophic forgetting) các phụ thuộc dài hạn. Xét câu:

> *"Tôi lớn lên ở Huế, ... (năm mươi từ xen giữa) ... nên tôi nói giọng ____."*

Để dự đoán từ cuối, mô hình phải dùng thông tin từ bước $t = 5$ tại bước $t = 60$. Gradient của mất mát tại $t = 60$ theo tham số ảnh hưởng bước $t = 5$ bị nhân với $\sigma_{\max}(\mathbf{W}_{hh})^{55}$. Với $\sigma_{\max} = 0.9$, hệ số là $0.9^{55} \approx 3.0 \times 10^{-3}$; với $\sigma_{\max} = 0.7$, là $0.7^{55} \approx 3.0 \times 10^{-9}$.

Điều tinh tế cần nhấn mạnh: mô hình *không phải* không có khả năng biểu diễn phụ thuộc đó. Trạng thái ẩn hoàn toàn đủ dung lượng mang thông tin "Huế" qua 55 bước. Vấn đề là **tín hiệu học** không truyền ngược được quãng đường đó, nên mô hình không bao giờ *khám phá ra* rằng nó nên mang thông tin ấy. Một lần nữa, đây là thất bại tối ưu hoá, không phải thất bại biểu diễn — đúng như sự phân biệt đặt ra ở Mục I..

V.6. Băng chuyền sai số hằng: suy diễn thay vì khẳng định

LSTM thay phép cập nhật *nhân* của (15) bằng một phép cập nhật *cộng* trên ô nhớ $\mathbf{c}_t$:

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t, \tag{19}$$

với $\mathbf{f}_t$ là cổng quên, $\mathbf{i}_t$ cổng vào và $\tilde{\mathbf{c}}_t$ ô nhớ ứng viên.

Lấy Jacobian đầy đủ của (19). Ba cổng đều là hàm của $\mathbf{h}_{t-1}$, mà $\mathbf{h}_{t-1} = \mathbf{o}_{t-1} \odot \tanh(\mathbf{c}_{t-1})$ lại phụ thuộc $\mathbf{c}_{t-1}$. Vậy nên

$$\frac{\partial \mathbf{c}_t}{\partial \mathbf{c}_{t-1}} = \underbrace{\text{diag}(\mathbf{f}_t)}_{\text{đường trực tiếp}} + \underbrace{\mathbf{R}_t}_{\text{đường qua các cổng}}, \tag{20}$$

trong đó

$$\mathbf{R}_t = \text{diag}(\mathbf{c}_{t-1}) \frac{\partial \mathbf{f}_t}{\partial \mathbf{c}_{t-1}} + \text{diag}(\tilde{\mathbf{c}}_t) \frac{\partial \mathbf{i}_t}{\partial \mathbf{c}_{t-1}} + \text{diag}(\mathbf{i}_t) \frac{\partial \tilde{\mathbf{c}}_t}{\partial \mathbf{c}_{t-1}}.$$

Lưu ý

$\mathbf{R}_t$ **không** bằng không và không được phép lặng lẽ bỏ đi. Nhiều trình bày phổ thông viết $\partial \mathbf{c}_t / \partial \mathbf{c}_{t-1} = \mathrm{diag}(\mathbf{f}_t)$ rồi kết luận ngay — đó là một xấp xỉ, không phải một đồng nhất thức. Lập luận đúng phải nêu điều kiện để $\mathbf{R}_t$ nhỏ.

Mỗi số hạng của $\mathbf{R}_t$ chứa một đạo hàm cổng, mà các cổng đều là sigmoid hoặc tanh của một hàm của $\mathbf{c}_{t-1}$ đi qua $\tanh(\mathbf{c}_{t-1})$. Khi cổng quên bão hoà mở — $\mathbf{f}_t \to \mathbf{1}$, tức tiền kích hoạt của nó nằm sâu ở nhánh dương — thì $\partial \mathbf{f}_t / \partial \mathbf{c}_{t-1} \to \mathbf{0}$ theo đúng (9); đồng thời nếu $\mathbf{c}_{t-1}$ đã lớn thì $\tanh'(\mathbf{c}_{t-1}) \to 0$ làm nhỏ cả hai số hạng còn lại. Trong chế độ đó $\mathbf{R}_t \approx \mathbf{0}$ và

$$\prod_{t=2}^{T} \frac{\partial \mathbf{c}_t}{\partial \mathbf{c}_{t-1}} \;\approx\; \prod_{t=2}^{T} \mathrm{diag}(\mathbf{f}_t) \;\approx\; \mathbf{I}. \tag{21}$$

Gradient đi qua T bước thời gian mà *không* chịu suy giảm cấp số nhân. Đây là *băng chuyền sai số hằng* (constant error carousel).

V.6.1. Vì sao RNN thuần không thể làm được điều tương tự

Đối chiếu với tích tương ứng của RNN thuần, $\prod_{t=2}^{T} \mathrm{diag}(1 - \mathbf{h}_{t-1}^2)\mathbf{W}_{hh}^\top$, câu hỏi tự nhiên là: có chọn được tham số nào khiến tích đó bằng $\mathbf{I}$ không? Câu trả lời là không, và lý do mang tính cấu trúc chứ không phải may rủi.

Đặt $\mathbf{W}_{hh} = \mathbf{I}$ — lựa chọn thuận lợi nhất có thể. Tích trở thành $\prod_t \mathrm{diag}(1 - \mathbf{h}_{t-1}^2)$. Nhưng $1 - h^2 < 1$ ngặt với mọi $h \neq 0$, nên tích này co lại nghiêm ngặt trừ khi $\mathbf{h}_t = \mathbf{0}$ với mọi t — tức mạng không mang thông tin nào cả. Ta gặp một lưỡng nan thực sự:

Lưỡng nan của RNN thuần

Hoặc trạng thái ẩn khác không — và khi đó $\mathrm{diag}(1 - \mathbf{h}_t^2)$ co chặt, gradient tiêu biến theo cấp số nhân; hoặc trạng thái ẩn bằng không — và khi đó mạng không biểu diễn gì. **Không tồn tại** bộ tham số nào cho phép RNN thuần vừa mang thông tin vừa truyền gradient không suy giảm qua nhiều bước. LSTM thoát khỏi lưỡng nan này bằng cách tạo một đường dẫn *cộng tính* đi vòng qua phi tuyến: trong (19), $\mathbf{c}_{t-1}$ vào thẳng $\mathbf{c}_t$ không qua một tanh nào.

Về mặt nguyên lý, (21) và kết nối tắt của ResNet là cùng một thủ pháp đặt ở hai trục khác nhau. ResNet biến Jacobian theo *chiều sâu* thành $\mathbf{I} + \mathcal{F}'$; LSTM biến Jacobian theo *thời gian* thành $\mathrm{diag}(\mathbf{f}_t) + \mathbf{R}_t$ với $\mathbf{f}_t \approx \mathbf{1}$. Trong cả hai trường hợp, một số hạng *bậc không* gần bằng $\mathbf{I}$ được cài vào tích, và chính số hạng đó gánh gradient đi xa. Xem Mục VII..

VI. Thực nghiệm: đo chuẩn gradient theo độ sâu tại bước khởi tạo

Mục III. chỉ cho một *chặn trên* (7). Mục này đo giá trị thực của gradient để thấy chặn đó chặt đến đâu, và kiểm tra khi nào tầng đầu thực sự không còn được cập nhật. Chỉ một cặp (hàm kích hoạt, sơ đồ khởi tạo) thay đổi giữa hai điều kiện; mọi thứ khác giữ nguyên.

VI.1. Giao thức

- **Kiến trúc.** L tầng tuyến tính: $L-1$ tầng ẩn rộng $D=64$ và một tầng ra một chiều; đầu vào 64 chiều. Hàm kích hoạt đặt sau mỗi tầng ẩn, không đặt sau tầng ra. Mọi độ lệch khởi tạo bằng 0.
- **Độ sâu.** $L \in \{5,\ 10,\ 20\}$.
- **Điều kiện A** — sigmoid, khởi tạo `nn.init.xavier_normal_` [4], $\mathrm{Var}(w) = 2/(n_{\text{in}} + n_{\text{out}})$.
- **Điều kiện B** — ReLU, khởi tạo `nn.init.kaiming_normal_(nonlinearity='relu')` [5], $\mathrm{Var}(w) = 2/n_{\text{in}}$. Cả L tầng (kể cả tầng ra) dùng cùng sơ đồ khởi tạo của điều kiện.
- **Dữ liệu và mất mát.** $N = 256$ mẫu, $\mathbf{x} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}_{64})$, $y \sim \mathcal{N}(0,1)$; `nn.MSELoss()` (trung bình trên lô).
- **Hạt giống.** $\{42,\ 100,\ 2024,\ 2026,\ 9999\}$. Với mỗi hạt giống, `random`, `numpy` và `torch` được đặt lại, dữ liệu được sinh *trước*, rồi mới khởi tạo mạng — nên A và B thấy cùng một lô dữ liệu. `torch.use_deterministic_algorithms(True)`, `float32`, CPU.
- **Phép đo.** Đo tại bước khởi tạo, một lượt lan truyền tiến và *một* lượt lan truyền ngược duy nhất; không có bước tối ưu nào.

Chuẩn gradient. Với $\mathbf{g}^{[l]} = \partial \mathcal{L}/\partial \mathbf{W}^{[l]}$ (chỉ trọng số, không tính độ lệch),

$$\left\|\mathbf{g}^{[l]}\right\|_F = \sqrt{\textstyle\sum_{i,j}\left(g_{ij}^{[l]}\right)^2} \qquad (\texttt{torch.linalg.norm(W.grad)}), \qquad \rho = \frac{\left\|\mathbf{g}^{[1]}\right\|_F}{\left\|\mathbf{g}^{[L]}\right\|_F}.$$

Chặn c trong bảng được *đo*, không giả định: $c = \max \sigma' \cdot \overline{\sigma_{\max}}$, với $\overline{\sigma_{\max}}$ là trung bình chuẩn phổ của $\mathbf{W}^{[2]}, \ldots, \mathbf{W}^{[L]}$ — các ma trận mà lượt ngược thực sự đi qua.

Tiêu chí đóng băng. So chuẩn tuyệt đối $\left\|\mathbf{g}^{[1]}\right\|_F$ với epsilon máy là sai tiêu chí: phép cộng `float32` $w - \eta g$ bị làm tròn về đúng w khi $|\eta g|$ nhỏ hơn khoảng nửa ulp của w, tức một đại lượng *tương đối* so với $|w|$. Vì vậy ta đo, với $\eta = 0.1$,

$$r = \frac{\eta \left\|\mathbf{g}^{[1]}\right\|_F}{\left\|\mathbf{W}^{[1]}\right\|_F} \quad \text{so với} \quad \epsilon_{32} = 2^{-23} \approx 1.19 \times 10^{-7},$$

và kiểm tra trực tiếp: tỉ lệ phần tử của $\mathbf{W}^{[1]}$ không đổi sau phép trừ $\mathbf{W}^{[1]} - \eta\,\mathbf{g}^{[1]}$, cùng với `torch.equal` (toàn bộ cập nhật bị mất).

Thực nghiệm đo gradient với PyTorch (`experiment.py`)

```
"""Section VI experiment: gradient norms at initialisation, 5 seeds x 3 depths.

L linear layers (L-1 hidden of width 64 + 1 output of width 1), biases = 0.
  A. Sigmoid + xavier_normal_
  B. ReLU    + kaiming_normal_(nonlinearity='relu')
One forward + backward pass at init, no optimizer step, float32, CPU.
```

```
Run: python experiment.py   (writes results.md next to this file)
"""
import random
import sys
from pathlib import Path

import numpy as np
import torch
import torch.nn as nn

SEEDS = [42, 100, 2024, 2026, 9999]
DEPTHS = [5, 10, 20]
D, N, LR = 64, 256, 0.1
EPS = torch.finfo(torch.float32).eps
CONDS = {"A: sigmoid + Xavier": "sigmoid", "B: ReLU + He": "relu"}

torch.use_deterministic_algorithms(True)
torch.set_default_dtype(torch.float32)


def seed_all(seed):
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)


def build(L, act):
    layers = []
    for l in range(L):
        lin = nn.Linear(D, 1 if l == L - 1 else D)
        if act == "sigmoid":
            nn.init.xavier_normal_(lin.weight)
        else:
            nn.init.kaiming_normal_(lin.weight, nonlinearity="relu")
        nn.init.zeros_(lin.bias)
        layers.append(lin)
        if l < L - 1:  # no activation after the output layer
            layers.append(nn.Sigmoid() if act == "sigmoid" else nn.ReLU())
    return nn.Sequential(*layers)


def run(seed, L, act):
    seed_all(seed)                      # same data for both conditions
    x, y = torch.randn(N, D), torch.randn(N, 1)
    net = build(L, act)
    nn.MSELoss()(net(x), y).backward()
    Ws = [m.weight for m in net if isinstance(m, nn.Linear)]
    g = [torch.linalg.norm(W.grad).item() for W in Ws]  # Frobenius, weights only
    # conservative bound c = max sigma' * sigma_max(W), averaged over the layers
    # the backward product passes through (k = 2..L); max sigma' = 1/4 or 1
    smax = np.mean([torch.linalg.matrix_norm(W.detach(), ord=2).item() for W in Ws
    [1:]])
    c = (0.25 if act == "sigmoid" else 1.0) * smax
    W1 = Ws[0].detach()
```

```
    r = LR * g[0] / torch.linalg.norm(W1).item()        # relative update of layer 1
    W1new = W1 - LR * Ws[0].grad
    frozen = torch.equal(W1new, W1)                     # update fully lost to
    rounding
    unchanged = (W1new == W1).float().mean().item()     # fraction of entries lost
    return {"g1": g[0], "gL": g[-1], "rho": g[0] / g[-1],
            "log10rho": float(np.log10(g[0] / g[-1])), "r": r, "frozen": frozen, "c"
    : c,
            "unchanged": unchanged}


def ms(vals):
    a = np.array(vals, dtype=np.float64)
    return f"{a.mean():.3g} \u00b1 {a.std(ddof=1):.3g}"


def main():
    # self-check: same seed twice -> identical numbers
    assert run(42, 5, "sigmoid") == run(42, 5, "sigmoid")
    assert run(42, 5, "relu") == run(42, 5, "relu")

    head = (f"torch {torch.__version__} | python {sys.version.split()[0]} | float32
    CPU | "
            f"seeds={SEEDS} | D={D} N={N} lr={LR} | eps32={EPS:.3g}")
    cols = ["condition", "L", "c (bound)", "||g[1]||_F", "||g[L]||_F", "rho = ||g
    [1]||/||g[L]||",
            "log10(rho)", "r = lr*||g[1]||/||W[1]||", "r/eps32", "W[1] entries
    unchanged", "frozen (of 5)"]
    rows = []
    for name, act in CONDS.items():
        for L in DEPTHS:
            res = [run(s, L, act) for s in SEEDS]
            col = lambda k: [d[k] for d in res]
            rows.append([name, str(L), ms(col("c")), ms(col("g1")), ms(col("gL")),
    ms(col("rho")),
                         ms(col("log10rho")), ms(col("r")),
                         ms([v / EPS for v in col("r")]), ms(col("unchanged")),
                         f"{sum(col('frozen'))}/{len(SEEDS)}"])
            for s, d in zip(SEEDS, res):
                print(f"  seed={s:5d} L={L:2d} {act:7s} g1={d['g1']:.4e} gL={d['gL
    ']:.4e} "
                      f"rho={d['rho']:.4e} r={d['r']:.4e} unch={d['unchanged']:.4f}
    frozen={d['frozen']}")

    table = ["| " + " | ".join(cols) + " |", "|" + "---|" * len(cols)]
    table += ["| " + " | ".join(r) + " |" for r in rows]
    print(head)
    print("\n".join(table))
    Path(__file__).with_name("results.md").write_text(
        "# Gradient norms at initialisation (mean +/- std, ddof=1, over 5 seeds)\n\n
    "
        f"`{head}`\n\n" + "\n".join(table) + "\n", encoding="utf-8")


```

```
105 if __name__ == "__main__":
106     main()
```

VI.2. Kết quả

Bảng 3 và Bảng 4 chép nguyên từ `results.md` do `experiment.py` sinh ra (PyTorch 2.14.0+cpu, CPython 3.12.3): trung bình ± độ lệch chuẩn mẫu (`ddof=1`) trên 5 hạt giống, ba chữ số có nghĩa. Chạy lại script cho kết quả trùng từng byte.

Bảng 3: Chặn c đo được và chuẩn gradient tại bước khởi tạo của MLP rộng 64 ($N = 256$ mẫu, $L \in \{5, 10, 20\}$), trung bình ± độ lệch chuẩn trên 5 hạt giống. Với sigmoid + Xavier, $\log_{10}\rho$ giảm gần tuyến tính theo L; với ReLU + He, nó nằm gần 0.

L	c (chặn)	$\left\|\mathbf{g}^{[1]}\right\|_F$	$\left\|\mathbf{g}^{[L]}\right\|_F$	ρ	$\log_{10}\rho$
A: sigmoid + Xavier					
5	0.449 ± 0.00761	$(3.96 \pm 1.15) \times 10^{-3}$	3 ± 2.16	$(2.18 \pm 1.75) \times 10^{-3}$	-2.78 ± 0.358
10	0.469 ± 0.00277	$(2.79 \pm 0.523) \times 10^{-6}$	4.93 ± 2.48	$(9.25 \pm 10.1) \times 10^{-7}$	-6.19 ± 0.373
20	0.479 ± 0.00293	$(1.59 \pm 0.62) \times 10^{-12}$	8.44 ± 3.2	$(2.28 \pm 1.63) \times 10^{-13}$	-12.7 ± 0.29
B: ReLU + He					
5	2.4 ± 0.0379	1.93 ± 0.75	10.9 ± 4.48	0.189 ± 0.0584	-0.74 ± 0.135
10	2.59 ± 0.0179	1.4 ± 0.571	9.55 ± 9.71	0.302 ± 0.25	-0.638 ± 0.358
20	2.68 ± 0.0141	4.08 ± 7.49	28.7 ± 57.6	0.462 ± 0.581	-0.53 ± 0.409

Bảng 4: Tiêu chí đóng băng tầng 1 với $\eta = 0.1$ (cùng thiết lập với Bảng 3). Cột cuối đếm số hạt giống mà `torch.equal` xác nhận toàn bộ cập nhật bị làm tròn mất: đóng băng hoàn toàn chỉ xảy ra ở sigmoid + Xavier, $L = 20$, ở 5/5 hạt giống.

L	$r = \eta\|\mathbf{g}^{[1]}\|_F / \|\mathbf{W}^{[1]}\|_F$	r/ϵ_{32}	Tỉ lệ phần tử $\mathbf{W}^{[1]}$ không đổi	Đóng băng hoàn toàn
A: sigmoid + Xavier				
5	$(4.93 \pm 1.45) \times 10^{-5}$	413 ± 122	0.00176 ± 0.00087	0/5
10	$(3.46 \pm 0.638) \times 10^{-8}$	0.291 ± 0.0535	0.678 ± 0.0553	0/5
20	$(1.98 \pm 0.764) \times 10^{-14}$	$(1.66 \pm 0.641) \times 10^{-7}$	1 ± 0	5/5
B: ReLU + He				
5	0.017 ± 0.00666	$(1.42 \pm 0.558) \times 10^{5}$	0 ± 0	0/5
10	0.0123 ± 0.00505	$(1.03 \pm 0.424) \times 10^{5}$	0 ± 0	0/5
20	0.0361 ± 0.0663	$(3.02 \pm 5.56) \times 10^{5}$	0 ± 0	0/5

VI.3. Phân tích

Điều kiện A (sigmoid + Xavier). $\log_{10}\rho$ giảm gần tuyến tính theo độ sâu: -2.78, -6.19, -12.7 ở $L = 5, 10, 20$. Quy về mỗi tầng, $10^{\log_{10}\rho/(L-1)}$ cho 0.202, 0.205, 0.215 — một hệ số suy giảm gần như không đổi, tức suy giảm theo hàm mũ đúng như chặn dự báo khi $c < 1$ (xem Hình 5 ở Phụ lục A.).

Tuy nhiên chặn đo được là $c \approx 0.45$–0.48, *không phải* 0.25: chuẩn phổ trung bình của các ma trận Xavier trong mạng, $c/0.25$, là khoảng 1.8–1.9 chứ không bằng 1. Hệ số suy giảm thực (≈ 0.2) còn nhỏ hơn c khoảng hai lần; khoảng cách đó phù hợp với hai yếu tố (a) bão hoà và (b) lệch hướng kỳ dị mà c bỏ qua (thí nghiệm này không tách riêng đóng góp của từng yếu tố). Nói cách khác, ở đây chặn đúng nhưng lỏng. (Vì $\mathbf{g}^{[l]}$ còn chứa thừa số kích hoạt $\mathbf{a}^{[l-1]}$, phép so sánh giữa ρ và c^{L-1} chỉ có ý nghĩa ở bậc độ lớn.)

Đóng băng ở điều kiện A phụ thuộc độ sâu, và chỉ xảy ra hoàn toàn ở $L = 20$. Với $L = 5$, $r/\epsilon_{32} = 413$: tầng 1 được cập nhật bình thường. Với $L = 10$, $r/\epsilon_{32} = 0.291 < 1$, nhưng chỉ 67.8% phần tử của $\mathbf{W}^{[1]}$ đứng yên và *không* hạt giống nào đóng băng hoàn toàn: r là trung bình trên cả ma trận, còn phép làm tròn xảy ra từng phần tử, nên các phần tử có $|g_{ij}|/|w_{ij}|$ lớn vẫn nhích được. Tầng 1 ở $L = 10$ bị đóng băng *một phần*. Chỉ ở $L = 20$ ($r/\epsilon_{32} \approx 1.66 \times 10^{-7}$) toàn bộ cập nhật bị làm tròn mất, ở cả 5/5 hạt giống. Vì r tỉ lệ thuận với η, tốc độ học nhỏ hơn 0.1 sẽ đẩy ngưỡng này xuống độ sâu nhỏ hơn.

Điều kiện B (ReLU + He). Chặn đo được là $c \approx 2.4$–$2.68 > 1$, vậy mà không có bùng nổ: ρ trung bình nằm trong $[0.189,\ 0.462]$, dưới 1. Đây là minh hoạ trực tiếp cho điểm đã nêu ở Mục III.: $c > 1$ chỉ cho phép bùng nổ, không bảo đảm nó. Điều He bảo toàn là *phương sai* tín hiệu (độ lợi trung bình bình phương ≈ 1), không phải $\sigma_{\max}$. Tầng 1 không đóng băng ở bất kỳ độ sâu nào ($r/\epsilon_{32} \sim 10^5$, không phần tử nào đứng yên).

Điều mà một lần chạy đơn lẻ không cho thấy là **độ phân tán tăng theo độ sâu**: độ lệch chuẩn của $\log_{10}\rho$ tăng từ 0.135 ($L = 5$) lên 0.409 ($L = 20$), và ở $L = 20$ độ lệch chuẩn của $\left\|\mathbf{g}^{[1]}\right\|_F$ (7.49) lớn hơn cả trung bình (4.08) — phân phối lệch mạnh, bị một vài hạt giống chi phối. Khởi tạo đúng giữ *bậc độ lớn trung bình*, nhưng không giữ được từng lần khởi tạo; đó là lý do cần báo cáo trung bình ± độ lệch chuẩn thay vì một con số.

VI.4. Phổ Jacobian đầu-cuối tại khởi tạo

Mục này đo phổ của $\mathbf{J} = \partial\mathbf{z}^{[L-1]}/\partial\mathbf{z}^{[1]}$ trong (10) (Mục III.5.) trên đúng các mạng, khởi tạo và hạt giống của thí nghiệm trước. Do tầng ra chỉ có một nơ-ron, $\mathbf{J}$ được tính trên chồng tầng ẩn 64×64 ($L-2$ thừa số), không tính tới đầu ra vô hướng. Với mỗi hạt giống ta lấy 16 mẫu đầu tiên của $\mathbf{x}$, tính $\mathbf{J}$ bằng `autograd` rồi lấy các giá trị kỳ dị; mạng được khởi tạo ở `float32` (trọng số trùng với Bảng 3) và chỉ phần đo phổ chuyển sang `float64`. Hạng số học là số $\sigma_i > 64\,\epsilon_{64}\sigma_{\max}$; $\log_{10}\kappa$ chỉ được báo khi hạng đủ 64, nếu không là n/a vì nhiễu làm tròn làm $\sigma_{\min}$ vô nghĩa. $\log_{10} c_J$ là chặn bảo thủ $\prod_{k=2}^{L-1} \|\mathbf{W}^{[k]}\|_2 \max_i |\sigma'(z_i^{[k-1]})|$ với σ' đo thực. Kết quả ở Bảng 5, chép từ `results_jacobian.md` (mỗi hạt giống lấy trung bình các $\log_{10}$ trên 16 mẫu, rồi trung bình ± độ lệch chuẩn mẫu trên 5 hạt giống).

Bảng 5: Phổ của Jacobian đầu-cuối $\mathbf{J} = \partial \mathbf{z}^{[L-1]}/\partial \mathbf{z}^{[1]}$ (chồng tầng ẩn 64×64, tầng ra loại bỏ) của MLP rộng 64 tại bước khởi tạo, $L \in \{5, 10, 20\}$, trung bình ± độ lệch chuẩn trên 5 hạt giống (16 mẫu mỗi hạt giống). Với sigmoid + Xavier cả phổ co lại theo L; với ReLU + He, RMS(σ) gần 1 nhưng hạng thiếu nhiều, nên phổ không đồng đều.

L	$\log_{10}\sigma_{\max}(\mathbf{J})$	$\log_{10}\mathrm{RMS}(\sigma)$	Hạng số học /64	$\log_{10}\kappa(\mathbf{J})$	$\log_{10} c_J$
A: sigmoid + Xavier					
5	-1.45 ± 0.0329	-1.93 ± 0.0165	64 ± 0	5.72 ± 0.811	-0.937 ± 0.0238
10	-4.39 ± 0.0528	-5.05 ± 0.024	64 ± 0	11.6 ± 0.781	-2.52 ± 0.0389
20	-10.6 ± 0.0672	-11.3 ± 0.0475	61 ± 0	n/a*	-5.63 ± 0.0249
B: ReLU + He					
5	0.557 ± 0.0219	-0.00502 ± 0.0252	28.9 ± 0.872	n/a*	1.32 ± 0.0237
10	0.688 ± 0.0872	-0.0286 ± 0.102	26.6 ± 2.56	n/a*	3.5 ± 0.039
20	0.676 ± 0.129	-0.122 ± 0.124	23.6 ± 1.09	n/a*	7.92 ± 0.0249

* n/a: ở 0/80 cặp (hạt giống, mẫu) $\mathbf{J}$ có đủ hạng số học, nên κ không được báo.

Xu hướng. Ở sigmoid + Xavier, *toàn bộ* phổ co lại theo độ sâu: cả $\log_{10}\sigma_{\max}$ lẫn $\log_{10}\mathrm{RMS}(\sigma)$ giảm cỡ nhiều bậc độ lớn từ $L = 5$ đến $L = 20$, cùng chiều với $\log_{10}\rho$ ở Bảng 3. Condition number tăng từ cỡ $10^{5.7}$ lên cỡ $10^{11.6}$, và ở $L = 20$ hạng số học chỉ còn 61 trên 64: một số giá trị kỳ dị đã rơi xuống dưới ngưỡng $64\,\epsilon_{64}\sigma_{\max}$ nên κ lớn hơn $1/(64\,\epsilon_{64}) \approx 7 \times 10^{13}$ và không còn đo được. Ở ReLU + He, $\log_{10}\mathrm{RMS}(\sigma)$ nằm gần 0 ở mọi độ sâu, nhưng hạng số học chỉ khoảng 24–29 trên 64 và $\log_{10}\sigma_{\max}$ dương.

Cơ chế. Với sigmoid, mỗi thừa số $\mathbf{W}^{[k]}\mathbf{D}^{[k-1]}$ có $\mathbf{D}$ bị chặn bởi 1/4 nên mọi giá trị kỳ dị cùng co lại, không chỉ giá trị lớn nhất. Với ReLU, $\mathbf{D}^{[k]}$ có phần tử bằng 0 (nơ-ron không hoạt động), mà $\mathrm{rank}(\mathbf{J}) \le \mathrm{rank}(\mathbf{D}^{[1]})$, nên một phần các giá trị kỳ dị bằng 0 về mặt số học, và hạng giảm dần khi L tăng, phù hợp với việc mỗi $\mathbf{D}^{[k]}$ thêm một ràng buộc hạng. Vì trung bình bình phương của cả 64 giá trị kỳ dị vẫn gần 1, phần còn lại phải lớn hơn 1 để bù, nên $\sigma_{\max} > 1$. Một ma trận thiếu hạng không thể có mọi $\sigma_i \approx 1$, nên mạng He ở đây không đạt dynamical isometry dù $\mathrm{RMS}(\sigma) \approx 1$. Số lượng nơ-ron không hoạt động cụ thể không được đo ở đây, nên không suy diễn thêm.

Liên hệ với chặn c. $\log_{10} c_J$ luôn lớn hơn $\log_{10}\sigma_{\max}(\mathbf{J})$ (kiểm tra bằng `assert` trên mọi mẫu), và khoảng cách rộng dần theo L ở cả hai điều kiện. Ở ReLU + He, c_J tăng theo L rất nhanh trong khi $\sigma_{\max}(\mathbf{J})$ gần như không đổi theo L, một minh hoạ trực tiếp cho việc $c > 1$ chỉ cho phép bùng nổ chứ không bảo đảm nó. Kết quả này đo ở bước khởi tạo, trên 5 hạt giống và 16 mẫu mỗi hạt giống (các mẫu cùng chung một bộ trọng số nên không độc lập), và không nói gì về lúc huấn luyện.

Cách đọc một bảng chuẩn gradient khi gỡ lỗi mô hình thật

Khi một mô hình sâu không giảm được mất mát huấn luyện, hãy in chuẩn gradient theo tầng, trên vài hạt giống, *trước khi* chỉnh tốc độ học hay thêm dữ liệu:

- **Giảm đơn điệu nhiều bậc độ lớn về phía đầu vào** ⇒ nhiều khả năng là triệt tiêu gradient. Kiểm tra hàm kích hoạt và sơ đồ khởi tạo trước tiên.
- **Tăng đơn điệu, hoặc xuất hiện `inf`/`NaN`** ⇒ bùng nổ gradient. Cắt ngưỡng gradient, giảm tốc độ học [1].
- **Mọi tầng cùng bậc độ lớn** ⇒ đường truyền gradient không phải nút thắt; nên tìm nguyên nhân ở chỗ khác (tốc độ học, dữ liệu, hàm mất mát).
- **Chuẩn bằng đúng 0 ở nhiều tầng với ReLU** ⇒ hiện tượng "ReLU chết"; cân nhắc LeakyReLU hoặc GELU.

Để biết tầng đầu có thực sự đứng yên hay không, đừng so chuẩn gradient với epsilon máy; hãy tính $r = \eta\|\mathbf{g}^{[1]}\|_F / \|\mathbf{W}^{[1]}\|_F$ và đếm số phần tử không đổi sau một bước cập nhật, như Bảng 4.

Lưu ý

Thực nghiệm đo gradient tại *bước khởi tạo*, chưa huấn luyện. Điều này là cố ý: nó tách biệt ảnh hưởng của kiến trúc và khởi tạo khỏi động lực học. Kết quả vì thế không nói gì về việc gradient thay đổi ra sao sau nhiều bước tối ưu; một mạng bắt đầu ở trạng thái đóng băng hoàn toàn (như điều kiện A, $L = 20$) chỉ thoát ra được nếu các tầng phía sau thay đổi đủ để nâng r lên trên ϵ_{32}.

VI.5. Động lực học trong quá trình huấn luyện: PlainNet so với ResNet

Các mục trước chỉ đo tại bước khởi tạo. Mục này theo dõi cùng các đại lượng *trong suốt* 10 epoch huấn luyện, và đặt residual connection cạnh mạng thuần để kiểm tra ba câu hỏi: (a) residual connection có cứu được sigmoid không; (b) ReLU + residual connection không có BatchNorm có ổn định không; (c) ở độ sâu lớn, BatchNorm cộng identity path chuyển thành độ chính xác ra sao.

Thiết lập. Dữ liệu là CIFAR-10: 10,000 ảnh huấn luyện (tập con cố định bằng hạt giống 0), đánh giá trên toàn bộ 10,000 ảnh test, chuẩn hoá theo kênh rồi duỗi thành vector 3072 chiều. Mạng là MLP rộng 256: tầng *stem* `Linear(3072, 256)` → Norm → Act, rồi L tầng ẩn `Linear(256, 256)`, rồi tầng *head* `Linear(256, 10)`. PlainNet xếp L khối `Linear`→Norm→Act; ResNet xếp $L/2$ khối

$$\mathbf{x}_{l+1} = \phi\big(\mathbf{x}_l + F(\mathbf{x}_l)\big), \qquad F = \texttt{Linear} \to \text{Norm} \to \phi \to \texttt{Linear} \to \text{Norm},$$

tức activation ϕ đặt *sau* phép cộng (post-addition), và cả hai kiến trúc có đúng $L + 2$ ma trận trọng số. Norm là `BatchNorm1d` hoặc `Identity` (khi có BatchNorm thì `bias=False`). Lưới thí nghiệm: 2 kiến trúc × $L \in \{8, 16, 32\}$ × 4 biến thể {Sigmoid + Xavier, Sigmoid + Xavier + BN, ReLU + Kaiming, ReLU + Kaiming + BN} × 3 hạt giống $\{42, 100, 2024\}$ = 72 lần chạy. Tối ưu bằng SGD, $\eta = 0.1$, không momentum, không weight decay, batch 128, 10 epoch, `float32` trên

CPU với `torch.use_deterministic_algorithms(True)`. Một lần chạy được coi là *phân kỳ* khi mất mát huấn luyện không còn hữu hạn; lần chạy đó dừng ngay.

Phép đo. Một *probe batch* cố định gồm 512 ảnh huấn luyện được đưa qua một lượt forward/backward riêng (chế độ train, không có bước tối ưu, buffer BatchNorm được khôi phục) tại epoch 0 (khởi tạo) và sau mỗi epoch. Gọi $\mathbf{g}_{\text{stem}}$ và $\mathbf{g}_{\text{head}}$ là gradient của $\mathcal{L}$ theo trọng số của stem và của head trên probe batch:

$$\mathbf{g}_{\text{stem}} = \frac{\partial \mathcal{L}}{\partial \mathbf{W}_{\text{stem}}}, \qquad \mathbf{g}_{\text{head}} = \frac{\partial \mathcal{L}}{\partial \mathbf{W}_{\text{head}}}, \qquad \text{stem gnorm} \;=\; \|\mathbf{g}_{\text{stem}}\|_F\,, \qquad \rho \;=\; \frac{\|\mathbf{g}_{\text{stem}}\|_F}{\|\mathbf{g}_{\text{head}}\|_F}.$$

Chỉ số chính và chỉ số phụ. Ta gọi

$$\text{TD} \;:=\; \log_{10} \frac{\|\mathbf{g}_{\text{first}}\|_F}{\|\mathbf{g}_{\text{last}}\|_F}$$

là *tổng suy giảm toàn mạng* (Total Degradation), chỉ số chính của tài liệu. TD đồng nhất với $\log_{10}\rho$ ở mọi bảng đã dùng, với $\mathbf{g}_{\text{first}}$ là gradient của tầng đầu (tầng 1 ở thí nghiệm khởi tạo, stem ở thí nghiệm huấn luyện) và $\mathbf{g}_{\text{last}}$ của tầng cuối (tầng L, head); tên TD chỉ để gọi cho gọn, không có số nào của các bảng trước bị đổi. TD < 0: gradient suy giảm về phía tầng đầu; TD > 0: khuếch đại. Chỉ số phụ *decay_slope* là hệ số góc của hồi quy bình phương tối thiểu của $\log_{10}\left\|\mathbf{g}^{[l]}\right\|_F$ theo chỉ số tầng l, tính trên mọi $L+2$ phần tử (stem, các khối, head) của thí nghiệm huấn luyện; đơn vị là số bậc độ lớn mỗi tầng, kèm R^2 của chính hồi quy đó. decay_slope > 0 nghĩa là gradient lớn dần về phía head (nhỏ dần về phía tầng đầu, tức vanishing). Phần tử bằng 0 hoặc không hữu hạn sẽ bị loại khỏi hồi quy; dữ liệu hiện có không có phần tử nào như vậy. Cả hai được tính từ `results_training.json` bằng `analysis_decay.py`, không huấn luyện thêm.

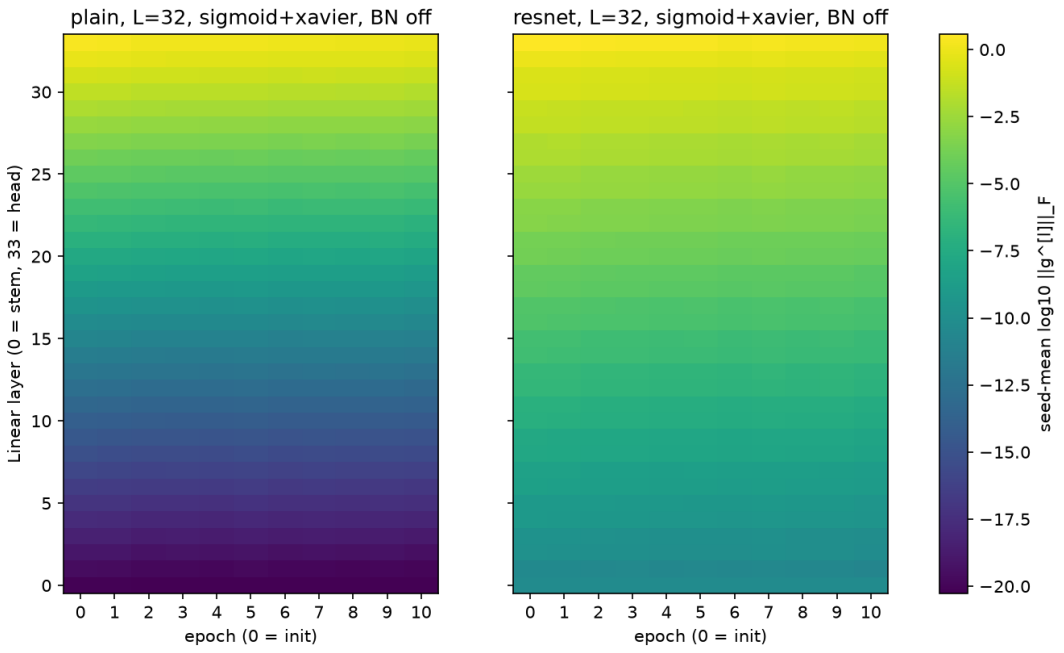
Ngoài ra, ở mỗi bước huấn luyện ta ghi cập nhật tương đối thực sự của stem, $r = \|\mathbf{W}_{t+1} - \mathbf{W}_t\|_F / \|\mathbf{W}_t\|_F$, và báo cáo trung vị theo epoch dưới dạng r/ϵ_{32}. Mọi số liệu dưới đây là trung bình $\pm$ độ lệch chuẩn mẫu (`ddof=1`) trên 3 hạt giống, lấy từ `results_training.json`.

Kết quả được trình bày theo thứ tự từ xu hướng đến cơ chế. Bảng 6 tóm tắt độ chính xác, $\log_{10}\rho$ và stem gnorm ở $L = 32$ với ReLU + Kaiming; Hình 3 cho thấy trường gradient theo tầng và theo epoch của sigmoid; Hình 4 cho thấy $\log_{10}\rho$ theo epoch ở cả ba độ sâu. Ba đoạn (a)–(c) sau đó giải thích cơ chế đằng sau từng xu hướng. Bảng 7, đặt sau phần (d), bổ sung TD, decay_slope và R^2 ở $L = 32$ cho cả bốn biến thể.

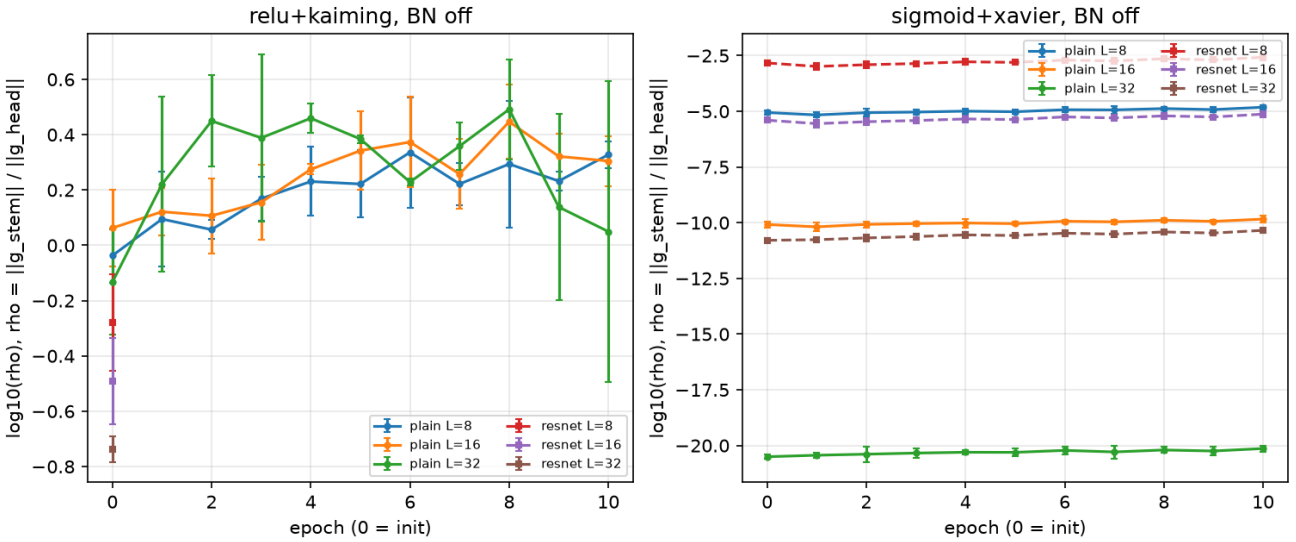
Bảng 6: Động lực học ở $L = 32$, ReLU + Kaiming, có và không có BatchNorm (MLP rộng 256, CIFAR-10 10,000 ảnh, SGD $\eta = 0.1$, 10 epoch, probe batch 512): độ chính xác test, $\log_{10}\rho$ và stem gnorm, trung bình ± độ lệch chuẩn trên 3 hạt giống. In đậm: độ chính xác test trung bình cao hơn, từ epoch 1, chỉ khi cả hai cùng không phân kỳ. Với BatchNorm, ResNet vượt PlainNet ở $L = 32$ từ epoch 1.

	Test acc (%)		$\log_{10}\rho$		Stem gnorm $\|\mathbf{g}_{\text{stem}}\|_F$	
Epoch	PlainNet	ResNet	PlainNet	ResNet	PlainNet	ResNet
ReLU + Kaiming + BatchNorm						
0	10.6 ± 0.962	10.0 ± 0.052	2.81 ± 0.0832	0.348 ± 0.101	626 ± 59.4	45.0 ± 1.39
1	13.1 ± 0.649	$\mathbf{18.3 \pm 4.03}$	-1.30 ± 0.0419	-0.914 ± 0.0523	0.0735 ± 0.00931	0.701 ± 0.0672
5	17.7 ± 0.396	$\mathbf{25.8 \pm 4.31}$	-1.36 ± 0.154	-0.569 ± 0.145	0.0531 ± 0.00477	0.993 ± 0.401
10	22.9 ± 1.09	$\mathbf{32.0 \pm 5.77}$	-1.26 ± 0.145	-0.342 ± 0.156	0.0469 ± 0.0076	1.07 ± 0.221
ReLU + Kaiming, không BatchNorm						
0	9.94 ± 0.0954	9.58 ± 0.37	-0.133 ± 0.191	-0.737 ± 0.0462	2.95 ± 0.9	$(2.32 \pm 0.244) \times 10^3$
1	$11.5 \pm 2.57^\dagger$	div.*	$0.221 \pm 0.317^\dagger$	div.*	$0.589 \pm 0.423^\dagger$	div.*
5	$18.9 \pm 0.0354^\dagger$	div.*	$0.384 \pm 0.0145^\dagger$	div.*	$0.891 \pm 0.503^\dagger$	div.*
10	$16.1 \pm 6.82^\dagger$	div.*	$0.0497 \pm 0.545^\dagger$	div.*	$5.41 \pm 3.08^\dagger$	div.*

* div.: 3/3 hạt giống phân kỳ ngay ở epoch 1, bước 1. † Trung bình trên 2 hạt giống: hạt giống 42 của PlainNet phân kỳ ở epoch 1, bước 3. Toàn lưới có 10/72 lần chạy phân kỳ (9 ResNet ReLU không BN ở cả ba độ sâu, 1 PlainNet ReLU không BN, $L = 32$); không lần chạy nào có BatchNorm hoặc sigmoid bị phân kỳ.



Hình 3: Heatmap của $\log_{10}\left\|\mathbf{g}^{[l]}\right\|_F$ (MLP rộng 256, CIFAR-10 10,000 ảnh, SGD $\eta = 0.1$; trung bình trên 3 hạt giống, thang màu viridis chung cho hai panel, từ khoảng -20 đến 0) ở $L = 32$, Sigmoid + Xavier, không BatchNorm. Trục hoành: epoch (0 = khởi tạo); trục tung: chỉ số tầng tuyến tính (0 = stem, 33 = head). Trái: PlainNet; phải: ResNet. Kết luận: residual connection kéo stem từ khoảng $10^{-20.5}$ lên khoảng $10^{-10.8}$, nhưng gradient vẫn giảm theo hàm mũ từ head về stem và không epoch nào thay đổi được điều đó.



Hình 4: $\log_{10}\rho$ theo epoch (0 = khởi tạo) của MLP rộng 256, CIFAR-10 10,000 ảnh, SGD $\eta = 0.1$, không BatchNorm; đường liền là PlainNet, đường đứt là ResNet, mỗi màu một độ sâu $L \in \{8, 16, 32\}$, thanh sai số là độ lệch chuẩn trên các hạt giống còn hữu hạn. Trái: ReLU + Kaiming; phải: Sigmoid + Xavier. Kết luận: với sigmoid, mọi đường gần như nằm ngang suốt 10 epoch, tức trong thiết lập này huấn luyện không làm thay đổi đáng kể vanishing gradient; với ReLU, ResNet không BN chỉ có điểm epoch 0 vì mọi lần chạy đều phân kỳ ở epoch 1.

(a) Sigmoid ResNet với activation sau phép cộng vẫn vanishing. *Cơ chế* (diễn giải lý thuyết). Đặt $\mathbf{s}_l = \mathbf{x}_l + F(\mathbf{x}_l)$. Jacobian của một khối là

$$\frac{\partial \mathbf{x}_{l+1}}{\partial \mathbf{x}_l} \;=\; \operatorname{diag}\big(\sigma'(\mathbf{s}_l)\big)\left(\mathbf{I} + \frac{\partial F}{\partial \mathbf{x}_l}\right), \qquad \sigma'(\cdot) \le \tfrac{1}{4}.$$

Identity path chỉ bảo vệ gradient *bên trong* dấu ngoặc. Thừa số $\operatorname{diag}(\sigma')$ nằm ngoài nên vẫn bị nhân một lần mỗi khối, và chặn (7) của Mục III. vẫn áp dụng, chỉ với số thừa số ít hơn. Trên đường ngắn nhất từ head về stem, PlainNet đi qua $L+1$ thừa số σ' (stem cộng L tầng ẩn), còn ResNet đi qua $L/2+1$ (stem cộng $L/2$ activation sau phép cộng). Lý thuyết vì vậy dự báo $\log_{10}\rho_{\text{ResNet}} / \log_{10}\rho_{\text{Plain}} \approx (L/2+1)/(L+1)$.

Bằng chứng. Tại khởi tạo, $\log_{10}\rho$ của PlainNet là -5.05, -10.1, -20.5 và của ResNet là -2.83, -5.40, -10.8 ở $L = 8, 16, 32$. Tỉ số đo được 0.560, 0.536, 0.526 khớp với dự báo $5/9 = 0.556$, $9/17 = 0.529$, $17/33 = 0.515$. Quy về mỗi thừa số σ', hệ số suy giảm là 0.23–0.27, sát chặn $1/4$, ở cả hai kiến trúc. Hình 3 cho thấy trường gradient theo tầng không đổi qua 10 epoch, và Hình 4 (phải) cho thấy $\log_{10}\rho$ chỉ nhích 0.24–0.45 sau 10 epoch. Ở $L = 32$, cập nhật tương đối của stem ResNet chỉ còn $r/\epsilon_{32} \approx 0.8$–$2.3 \times 10^{-8}$ (PlainNet: đúng 0, mọi bước đều bị `torch.equal` xác nhận không đổi). Độ chính xác test của cả 18 lần chạy Sigmoid không BN (6 cấu hình) ở epoch 10 đều là 10.0%, tức mức đoán ngẫu nhiên.

Kết luận. Residual connection xấp xỉ chia đôi số mũ của vanishing gradient nhưng không xoá nó. Điều này gợi ý rằng muốn identity path thực sự là đường cao tốc, activation bão hoà nên nằm *trong* nhánh F, không đặt sau phép cộng; thiết kế đó không được thí nghiệm ở đây.

(b) ReLU ResNet không BatchNorm phân kỳ ngay epoch đầu. *Cơ chế* (diễn giải lý thuyết). Không có BatchNorm, nhánh F với khởi tạo Kaiming giữ phương sai của đầu vào thay vì triệt tiêu nó, và phép cộng của identity path cộng dồn phương sai qua từng khối:

$$\mathrm{Var}(\mathbf{x}_{l+1}) \;\approx\; \mathrm{Var}(\mathbf{x}_l) + \mathrm{Var}\left(F(\mathbf{x}_l)\right) \;=\; (1+\kappa)\,\mathrm{Var}(\mathbf{x}_l) \;\Longrightarrow\; \mathrm{Var}(\mathbf{x}_{L/2}) \approx (1+\kappa)^{L/2}\,\mathrm{Var}(\mathbf{x}_0),$$

với $\kappa = \mathrm{Var}(F(\mathbf{x}_l))/\,\mathrm{Var}(\mathbf{x}_l)$ cỡ 1. Phương sai của forward pass tăng theo hàm mũ của số khối, kéo theo logits và gradient. PlainNet + Kaiming nhân phương sai với khoảng 1 mỗi tầng nên không cộng dồn.

Bằng chứng. Phương sai forward không được đo trực tiếp, nhưng $\mathbf{g}_{\text{head}} = \boldsymbol{\delta}_{\text{out}}\mathbf{a}^\top$ tỉ lệ với độ lớn activation cuối. Tại khởi tạo, $\|\mathbf{g}_{\text{head}}\|_F$ của ResNet là 51.8, 313, 1.26×10^4 ở $L = 8, 16, 32$ (4, 8, 16 khối), tức tăng khoảng 1.57–1.59 lần mỗi khối về chuẩn, hay khoảng 2.5 lần về phương sai. PlainNet cùng cấu hình giữ ở 2.4–4.3. Mất mát trên probe batch lúc khởi tạo của ResNet là 9.90, 41.8, 1480, so với $\ln 10 \approx 2.30$ của một bộ phân loại ngẫu nhiên. Stem gnorm ở $L = 32$ là $(2.32 \pm 0.244) \times 10^3$ (Bảng 6), nên bước SGD đầu tiên có $\eta\|\mathbf{g}_{\text{stem}}\|_F \approx 232$. Cả 9/9 lần chạy phân kỳ ở epoch 1, và mạng càng sâu thì càng sớm: bước 3–4 ($L = 8$), bước 2 ($L = 16$), bước 1 ($L = 32$). Trong Hình 4 (trái), các đường ResNet chỉ còn điểm epoch 0. PlainNet ReLU không BN cũng không hoàn toàn an toàn: 1/3 hạt giống ở $L = 32$ phân kỳ (epoch 1, bước 3).

Kết luận. Số liệu phù hợp với cách đọc rằng identity path chuyển phép *nhân* của mạng thuần thành phép *cộng* của phương sai, và không có cơ chế chuẩn hoá thì phép cộng đó cộng dồn thành bùng nổ. Phương sai forward không được đo, nên đây là diễn giải.

(c) Ở $L = 32$, BatchNorm + identity path đi cùng ρ gần 1 và độ chính xác cao hơn. *Cơ chế* (diễn giải lý thuyết). BatchNorm ở cuối nhánh F ép $\mathrm{Var}(F(\mathbf{x}_l)) \approx 1$ bất kể l (với $\gamma = 1$, $\beta = 0$ lúc khởi tạo), nên phương trình ở (b) đổi từ nhân sang cộng:

$$\mathrm{Var}(\mathbf{x}_{l+1}) \approx \mathrm{Var}(\mathbf{x}_l) + 1 \;\Longrightarrow\; \mathrm{Var}(\mathbf{x}_l) \approx \mathrm{Var}(\mathbf{x}_0) + l.$$

Phương sai tăng tuyến tính chứ không theo hàm mũ, và tỉ trọng của nhánh F so với identity path giảm cỡ $1/\sqrt{l}$. Vì vậy mỗi thừa số $\mathbf{I} + \partial F/\partial \mathbf{x}_l$ gần $\mathbf{I}$, và tích của chúng không trôi theo hàm mũ. PlainNet + BN không có identity path, nên gradient đi về stem phải qua $L + 1 = 33$ cặp BN–ReLU nối tiếp và không có gì giữ chúng gần nhau.

Bằng chứng (Bảng 6, phần trên).

- ρ **của ResNet ở gần** 1**.** Qua 11 lần đo, $\log_{10}\rho$ trung bình của ResNet nằm trong $[-0.914,\ 0.348]$, tức $\rho \in [0.122,\ 2.23]$. PlainNet bùng nổ về phía stem lúc khởi tạo ($\log_{10}\rho = 2.81$, $\rho \approx 643$, stem gnorm 626), rồi rơi xuống -1.30 sau epoch 1 và ở lại quanh -1.2 đến -1.4 ($\rho \approx 0.05$).
- **Stem của ResNet được cập nhật mạnh hơn nhiều.** Ở epoch 10, stem gnorm là 1.07 ± 0.221 (ResNet) so với 0.0469 ± 0.0076 (PlainNet), và cập nhật tương đối r/ϵ_{32} là $(5.08 \pm 0.46) \times 10^4$ so với 423 ± 76.4, tức cập nhật tương đối của stem ResNet lớn hơn khoảng 120 lần.
- **Độ chính xác tương ứng.** Test acc ở epoch 10 là $32.0 \pm 5.77\%$ (ResNet) so với $22.9 \pm 1.09\%$ (PlainNet). Theo từng hạt giống, ResNet đạt 38.5, 29.9, 27.5% và PlainNet đạt 24.2, 22.5, 22.2%, nên mọi hạt giống ResNet đều vượt mọi hạt giống PlainNet.

Lợi thế này chỉ xuất hiện ở độ sâu lớn. Ở $L = 8$ và $L = 16$, PlainNet + BN nhỉnh hơn (35.1 ± 4.10 so với $31.6 \pm 5.72\%$; 29.5 ± 2.88 so với $28.2 \pm 6.75\%$). Độ chính xác của PlainNet giảm đơn điệu theo L ($35.1 \to 29.5 \to 22.9\%$), còn ResNet gần như giữ nguyên ($31.6 \to 28.2 \to 32.0\%$).

Kết luận. Số liệu phù hợp với cách đọc rằng BatchNorm chặn sự cộng dồn phương sai và identity path giữ đường gradient về stem. Trong lưới 2×2 ở $L = 32$ (có/không BN, có/không identity path), chỉ tổ hợp có *cả hai* cho độ chính xác test cao nhất (ResNet không BN phân kỳ, PlainNet không BN thấp hơn), và tổ hợp đó đi cùng khoảng cách 9 điểm phần trăm so với PlainNet + BN. Đây là mô tả, không phải phân tích trung gian hay kiểm định thống kê; 3 hạt giống trong một cấu hình không độc lập.

(d) Khởi tạo chỉ đảm bảo ở $t = 0$, và BatchNorm ở mạng thuần sâu không miễn nhiễm. *Cơ chế* (diễn giải lý thuyết, không đo trực tiếp). Xavier và He chỉ cân bằng phương sai tại bước khởi tạo, dưới các giả thiết (G1)–(G4) của Mục III.; huấn luyện làm phân phối trọng số trôi đi nên các giả thiết đó không còn đảm bảo. BatchNorm không thay đổi điều này: lý thuyết trường trung bình của Yang và cộng sự [9] dự báo rằng ở mạng BatchNorm thuần không có kết nối tắt, gradient tại khởi tạo tăng theo hàm mũ của độ sâu.

Bằng chứng (số trong `results_training.md`). PlainNet ReLU + Kaiming + BN có $\log_{10} \rho$ tại epoch 0 là 0.862 ± 0.0482, 1.56 ± 0.0611, 2.81 ± 0.0832 ở $L = 8, 16, 32$, tức $\rho > 1$ ngay từ khởi tạo và $\log_{10} \rho$ tăng gần tuyến tính theo L; stem gnorm tương ứng là 8.22, 36.8, 626. Cùng mạng ReLU + Kaiming *không* BN có $\log_{10} \rho$ tại epoch 0 là -0.0372, 0.0626, -0.133, gần 0 (He hoạt động đúng ở $t = 0$), và stem gnorm ở $L = 32$ chỉ là 2.95 ± 0.9 so với 626 ± 59.4 khi có BN (Bảng 6). Sau epoch 1, $\log_{10} \rho$ của PlainNet + BN là 0.0232 ± 0.00475, -0.297 ± 0.0768, -1.3 ± 0.0419; sau epoch 10 là 0.332 ± 0.0716, -0.0106 ± 0.144, -1.26 ± 0.145. Đối chiếu: ResNet + BN giữ $\log_{10} \rho$ trong $[-0.914,\ 0.348]$ ở cả ba độ sâu tại các epoch $0, 1, 5, 10$. Một quan sát khác: Sigmoid + Xavier + BN PlainNet có $\log_{10} \rho$ tại epoch 0 là -0.16, -0.0732, -0.0524, tức không thấy bùng nổ; đây chỉ là một quan sát, không rút kết luận nào về lý thuyết của [9] cho sigmoid.

Kết luận (có phạm vi). Trong thiết lập này (MLP rộng 256, CIFAR-10 10,000 ảnh, SGD $\eta = 0.1$, 3 hạt giống), ReLU + Kaiming + BN không có kết nối tắt cho gradient ở stem lớn hơn hẳn ở head tại khởi tạo, và mức lệch tăng theo độ sâu; điều này phù hợp với dự báo của [9]. Số liệu không đủ để nói BatchNorm nói chung gây bùng nổ, và tài liệu này không so sánh với khởi tạo trực giao.

Bảng 7: Tổng suy giảm toàn mạng TD (đồng nhất với $\log_{10}\rho$), decay_slope (bậc độ lớn mỗi tầng; dương: gradient lớn dần về phía head) và R^2 của hồi quy đó, ở $L = 32$, epoch 0 và 10 (MLP rộng 256, CIFAR-10 10,000 ảnh, SGD $\eta = 0.1$, probe batch 512; trung bình ± độ lệch chuẩn trên 3 hạt giống). Ở Sigmoid + Xavier không BN, R^2 gần 1; ở các ô còn lại R^2 thấp, trừ PlainNet ReLU + BN ở epoch 0.

	TD		decay_slope		R^2	
Epoch	PlainNet	ResNet	PlainNet	ResNet	PlainNet	ResNet
ReLU + Kaiming + BatchNorm						
0	2.81 ± 0.0832	0.348 ± 0.101	-0.0792 ± 0.00161	-0.0239 ± 0.000603	0.978 ± 0.00236	0.402 ± 0.0194
10	-1.26 ± 0.145	-0.342 ± 0.156	0.00797 ± 0.00121	-0.00728 ± 0.00231	0.196 ± 0.0597	0.127 ± 0.0757
ReLU + Kaiming, không BatchNorm						
0	-0.133 ± 0.191	-0.737 ± 0.0462	0.0111 ± 0.00588	0.0293 ± 0.00214	0.576 ± 0.203	0.934 ± 0.015
10	$0.0497 \pm 0.545^{\dagger}$	n/a*	$0.000233 \pm 0.00769^{\dagger}$	n/a*	$0.38 \pm 0.199^{\dagger}$	n/a*
Sigmoid + Xavier + BatchNorm						
0	-0.0524 ± 0.0885	-0.242 ± 0.078	-0.00678 ± 0.000826	-0.00732 ± 0.00112	0.177 ± 0.0408	0.131 ± 0.0178
10	0.0398 ± 0.157	-0.0447 ± 0.131	-0.00385 ± 0.00171	-0.0073 ± 0.00171	0.186 ± 0.163	0.324 ± 0.173
Sigmoid + Xavier, không BatchNorm						
0	-20.5 ± 0.101	-10.8 ± 0.103	0.627 ± 0.00313	0.325 ± 0.000483	$1 \pm 3.31 \times 10^{-5}$	0.998 ± 0.000174
10	-20.1 ± 0.132	-10.3 ± 0.0984	0.624 ± 0.00149	0.325 ± 0.00241	$1 \pm 8.52 \times 10^{-5}$	0.997 ± 0.000608

* n/a: 3/3 hạt giống ResNet phân kỳ ngay ở epoch 1. † Trung bình trên 2 hạt giống (hạt giống 42 của PlainNet phân kỳ ở epoch 1).

Đọc decay_slope cùng R^2. Ở Sigmoid + Xavier không BN, $R^2 \approx 1$ ở cả hai epoch (Bảng 7): $\log_{10}\|\mathbf{g}\|$ gần tuyến tính theo tầng, và decay_slope của ResNet xấp xỉ một nửa của PlainNet (0.325 so với 0.627 ở epoch 0), cùng chiều với TD ở phần (a). Ở các biến thể có BatchNorm, R^2 thấp: gradient không đơn điệu theo tầng, nên một đường thẳng không mô tả được hình dạng của nó. Ví dụ, ở PlainNet ReLU + BN, hạt giống 42, epoch 10, chuẩn gradient gần như phẳng ở các tầng giữa trong khi stem thấp hơn hẳn tầng kế cận và head cao hơn hẳn các tầng giữa. Ngoại lệ là PlainNet ReLU + BN ở epoch 0, nơi $R^2 = 0.978$ và decay_slope âm, tức gradient lớn dần về phía stem. Vì vậy decay_slope chỉ là xu hướng tuyến tính tổng quát, và khi R^2 thấp thì độ lớn của nó không nên đọc như một tốc độ suy giảm; nó không thay thế TD, vốn chỉ so tầng đầu với tầng cuối.

Lưu ý

Số liệu chỉ có 3 hạt giống và 10 epoch trên 10,000 ảnh, nên các khoảng ± của ResNet (4–7 điểm phần trăm) còn rộng. Hai biến thể Sigmoid + BN không được đưa vào Bảng 6; ở $L = 32$, chúng đạt $18.9 \pm 3.55\%$ (PlainNet) và $22.3 \pm 6.47\%$ (ResNet) ở epoch 10, với $\log_{10}\rho$ trung bình của cả hai ở trong $[-0.41,\ 0.09]$ suốt quá trình huấn luyện. BatchNorm một mình đã xoá vanishing gradient của sigmoid về mặt chuẩn gradient, nhưng không đưa độ chính xác lên ngang ReLU + BN.

VI.6. Hạn chế của nghiên cứu

(1) **Tối ưu hoá.** Chỉ dùng SGD thuần (momentum 0, weight decay 0) với tốc độ học $\eta = 0.1$ *cố định* cho mọi cấu hình. Các kết luận về đóng băng và phân kỳ vì thế phụ thuộc η (chẳng hạn $r = \eta\|\mathbf{g}\|/\|\mathbf{W}\|$), và ảnh hưởng của kiến trúc chưa được tách khỏi ảnh hưởng của η.

Khảo sát quét tốc độ học (ví dụ $\eta \in \{0.01, 0.03, 0.1\}$) là hướng mở ngoài phạm vi của tài liệu; không được thực hiện và không có số liệu.

(2) **Số hạt giống.** 5 hạt giống ở thí nghiệm khởi tạo, 3 hạt giống ở thí nghiệm huấn luyện; các khoảng ± khá rộng, và 10/72 lần chạy phân kỳ nên một số ô chỉ có $n < 3$.

(3) **Phạm vi thiết lập.** 10 epoch, tập con 10,000 ảnh của CIFAR-10, MLP rộng 256 (thí nghiệm huấn luyện) hoặc 64 (thí nghiệm khởi tạo), độ sâu $L \in \{8, 16, 32\}$ và $\{5, 10, 20\}$. Không có kết luận nào cho CNN, Transformer, tập dữ liệu đầy đủ hay độ sâu khác.

(4) **Phép đo.** Gradient được đo trên một probe batch 512 ảnh, tại epoch 0 và sau mỗi epoch (các bảng chỉ ghi epoch $0, 1, 5, 10$). Phương sai forward không được đo trực tiếp, nên các giải thích cơ chế ở (a)–(d) là diễn giải, không phải kết quả đo.

(5) **Phương pháp chưa thử.** Không có thí nghiệm với Adam/AdamW và với khởi tạo trực giao.

(6) **Thống kê.** Chỉ báo trung bình ± độ lệch chuẩn trên số hạt giống nhỏ, không kiểm định thống kê; các nhận xét về hai đại lượng đi cùng nhau là mô tả, không phải kiểm định.

Adam: thảo luận lý thuyết, không phải quan sát. Tài liệu không chạy Adam, nên không có quan sát nào kèm theo đoạn này. Cập nhật Adam cho toạ độ i của tham số là

$$\Delta\theta_i \;=\; -\,\eta\,\frac{\hat{m}_i}{\sqrt{\hat{v}_i}+\epsilon_{\mathrm{A}}}, \qquad \epsilon_{\mathrm{A}} = 10^{-8},$$

với $\hat{m}_i$ và $\hat{v}_i$ là ước lượng moment bậc một và bậc hai của gradient đã hiệu chỉnh độ lệch. Phép so sánh với ϵ_{A} phải làm *từng phần tử*: với mỗi toạ độ i, so $\sqrt{\hat{v}_i}$ với ϵ_{A}; không dùng chuẩn Frobenius của gradient để so với ϵ_{A}, vì cập nhật Adam là theo từng toạ độ. Khi $\sqrt{\hat{v}_i} \gg \epsilon_{\mathrm{A}}$, bước xấp xỉ $\eta\,\hat{m}_i/\sqrt{\hat{v}_i}$ và gần như không phụ thuộc độ lớn của gradient. Khi $\sqrt{\hat{v}_i}$ cỡ hoặc nhỏ hơn ϵ_{A}, thì ϵ_{A} chiếm ưu thế ở mẫu số và bước co lại theo $\hat{m}_i/\epsilon_{\mathrm{A}}$. Do đó, nếu gradient của một số toạ độ ở tầng đầu có độ lớn cỡ ϵ_{A} hoặc nhỏ hơn, thì bước cập nhật của các toạ độ đó co lại theo cách vừa nêu; còn ở các toạ độ có $\sqrt{\hat{v}_i} \gg \epsilon_{\mathrm{A}}$, độ lớn bước không còn bị tỉ lệ theo độ lớn gradient. Đoạn này không khẳng định Adam tốt hơn hay kém hơn SGD trong thiết lập của tài liệu.

VII. Tổng kết và các nguyên lý khắc phục

Phần lớn lập luận của tài liệu xoay quanh một chặn trên, (7):

$$\left\|\boldsymbol{\delta}^{[l]}\right\| \le c^{\,L-l}\left\|\boldsymbol{\delta}^{[L]}\right\|, \qquad c = \sigma_{\max}(\mathbf{W})\cdot\max\sigma'.$$

Chặn này chỉ nói một chiều: $c < 1$ đủ để gradient co theo hàm mũ, còn $c \ge 1$ không bảo đảm điều gì về giá trị thực (Mục VI. đo được $c \approx 2.4$–2.68 ở điều kiện ReLU + He mà không có bùng nổ). Các kỹ thuật thực hành dưới đây có thể đọc như những cách nới hoặc tránh chặn đó — một góc nhìn hữu ích, không phải một định lý hay một phân loại vét cạn: khởi tạo trực giao theo hướng Dynamical Isometry (Mục III.5.) là một hướng nữa, không nằm trong bảng. Bảng 8 tóm tắt từng nguyên nhân cùng nguyên lý khắc phục tương ứng.

Bảng 8: Phân loại nguyên nhân biến đổi gradient và nguyên lý khắc phục tương ứng, đọc qua chặn trên $c = \sigma_{\max}(\mathbf{W}) \cdot \max \sigma'$. Đọc bảng theo hàng: mỗi nguyên lý nhắm vào đúng một thừa số của c (hoặc vào cấu trúc tích), nên chúng bổ sung chứ không thay thế nhau.

Nguyên nhân	Biểu hiện toán học	Kiến trúc bị ảnh hưởng	Nguyên lý khắc phục
Bão hoà hàm kích hoạt	$\max \sigma' = \frac{1}{4}$ (sigmoid), $\to 0$ khi $\lvert z \rvert$ lớn	MLP sâu, CNN đời đầu, RNN dùng tanh	Thay bằng hàm không bão hoà: ReLU, LeakyReLU, GELU $\Rightarrow$ nâng $\max \sigma'$ lên 1
Bán kính phổ lệch khỏi 1	$\prod_k \sigma_{\max}(\mathbf{W}^{[k]})$ phân kỳ hoặc tiêu biến	Mọi mạng sâu; nhạy nhất ở mạng rất sâu	Khởi tạo có kiểm soát phương sai: Xavier, He $\Rightarrow$ đặt $\sigma_{\max}(\mathbf{W}) \approx 1$
Tiền kích hoạt trôi vào vùng bão hoà	$\mathbf{z}$ dịch xa 0 theo chiều sâu, kéo $\sigma' \to 0$	Mạng sâu huấn luyện lâu, dịch chuyển hiệp biến nội	Chuẩn hoá: BatchNorm, LayerNorm $\Rightarrow$ tái định tâm $\mathbf{z}$ quanh 0
Cấu trúc tích thuần tuý của Jacobian	$\boldsymbol{\delta}^{[l]}$ là tích $L - l$ thừa số, không có số hạng bậc không	Mạng rất sâu (trên 50 tầng)	Kết nối tắt: Jacobian thành $\mathbf{I} + \mathbf{F}'$ $\Rightarrow$ tích có thêm số hạng $\mathbf{I}$
Chia sẻ trọng số theo thời gian	Chặn là $\sigma_{\max}(\mathbf{W}_{hh})^{T-1}$, không có trung bình hoá	RNN thuần	Cổng cộng tính: LSTM, GRU $\Rightarrow$ đường dẫn gần đồng nhất; cắt ngưỡng gradient cho chiều bùng nổ

VII.1. Bốn nguyên lý, đọc qua lăng kính của c

ReLU và GELU — nâng $\max \sigma'$. Với $\mathrm{ReLU}(z) = \max(0, z)$, đạo hàm bằng 1 trên toàn miền dương thay vì bị chặn bởi 1/4. Thừa số $\max \sigma'$ trong c nhảy từ 0.25 lên 1, tức chặn trên không còn ép gradient co bốn lần mỗi tầng; hệ số thực còn tuỳ tỉ lệ nơ-ron hoạt động. Giá phải trả là nơ-ron chết ($\sigma' = 0$ vĩnh viễn trên nhánh âm), điều mà LeakyReLU và GELU khắc phục bằng cách giữ một độ dốc nhỏ khác không. Trong thực nghiệm ở Mục VI. (cùng với khởi tạo He), cặp này giữ $\log_{10} \rho$ trong khoảng -0.74 đến -0.53 ở $L \leq 20$, so với -12.7 của sigmoid + Xavier ở $L = 20$ — đó là kết quả tại bước khởi tạo, trên một kiến trúc cụ thể, không phải một đảm bảo chung.

Xavier và He — điều khiển $\sigma_{\max}(\mathbf{W})$. Hai sơ đồ khởi tạo chọn phương sai trọng số sao cho phương sai tín hiệu được bảo toàn khi đi qua tầng. Chúng điều khiển độ lợi *trung bình bình phương*, không phải $\sigma_{\max}(\mathbf{W})$: với ma trận 64×64, thực nghiệm đo được $\sigma_{\max} \approx 1.8$–$1.9$ (Xavier) và chặn $c \approx 2.4$–2.68 (He), tức chặn trên c không về 1 dù gradient thực không bùng nổ. Xavier [4] dung hoà ràng buộc tiến và ngược bằng $\mathrm{Var}(w) = 2/(n_{\text{in}} + n_{\text{out}})$; He [5] dùng $\mathrm{Var}(w) = 2/n_{\text{in}}$, với hệ số 2 bù cho việc ReLU triệt tiêu một nửa số nơ-ron. Đây là một trong những can thiệp rẻ nhất: nó không tốn thêm phép tính nào sau bước khởi tạo.

BatchNorm và LayerNorm — nhắm giữ z ngoài vùng bão hoà, nhưng không có đảm bảo. Khởi tạo He hay Xavier chỉ đảm bảo phân phối tiền kích hoạt ở bước $t = 0$; khi

huấn luyện tiến triển, phân phối của $\mathbf{z}$ trôi đi và có thể trượt vào vùng bão hoà, kéo $\max \sigma'$ xuống. Chuẩn hoá [6] nhằm duy trì điều kiện đó trong huấn luyện bằng cách tái định tâm và tái co giãn $\mathbf{z}$ ở mỗi bước, nhưng *không có gì đảm bảo* nó giữ được sự ổn định, và chỉ đáng tin trong phạm vi các giả thiết của nó (lô đủ lớn với BatchNorm, tham số co giãn học được không trôi quá xa). Ở mạng thuần rất sâu, lý thuyết trường trung bình của Yang và cộng sự [9] dự báo chính BatchNorm có thể gây bùng nổ gradient tại khởi tạo. Số liệu ở Mục VI.5., phần (d), phù hợp với dự báo đó: PlainNet ReLU + Kaiming + BN có $\log_{10} \rho$ tại epoch 0 là 0.862, 1.56, 2.81 ở $L = 8, 16, 32$, và ở $L = 32$ stem gnorm là 626 so với 2.95 khi không có BN (Bảng 6). Kết quả này chỉ thuộc thiết lập MLP rộng 256, CIFAR-10 10,000 ảnh, SGD $\eta = 0.1$, 3 hạt giống, nên không đủ để nói về BatchNorm nói chung. Một hướng khác là khởi tạo trực giao theo Dynamical Isometry [8], nhắm vào phổ của cả Jacobian đầu-cuối (Mục III.5.); tài liệu này không thực nghiệm với nó, chỉ đo phổ tại khởi tạo ở Bảng 5. Trong nhóm ReLU + Kaiming ở $L = 32$ của Bảng 6, chỉ ResNet + BN vừa giữ ρ gần 1 vừa để stem học được, với độ chính xác test cao nhất trong bảng; vì không so sánh với khởi tạo trực giao, ta không kết luận đó là phương án rẻ nhất hay bền nhất.

Kết nối tắt — phá vỡ cấu trúc tích. Ba biện pháp trên đều cố đưa c về 1, nhưng vẫn chấp nhận rằng gradient là một *tích*. ResNet [7] tấn công chính cấu trúc đó. Với khối $\mathbf{a}^{[l]} = \mathbf{a}^{[l-1]} + \mathcal{F}(\mathbf{a}^{[l-1]})$, Jacobian giữa hai tầng trở thành

$$\frac{\partial \mathbf{a}^{[l]}}{\partial \mathbf{a}^{[l-1]}} = \mathbf{I} + \mathcal{F}'(\mathbf{a}^{[l-1]}).$$

Tích của các ma trận dạng $\mathbf{I} + \mathcal{F}'$ khai triển thành $\mathbf{I} + \sum_k \mathcal{F}'_k +$ (các số hạng bậc cao): luôn có một số hạng *bậc không* bằng đúng $\mathbf{I}$. Số hạng này không suy giảm theo chiều sâu; tổng vẫn có thể nhỏ hơn nếu các số hạng khác triệt tiêu một phần với nó, nhưng khi các nhánh $\mathcal{F}'_k$ nhỏ lúc khởi tạo thì đường tắt chiếm ưu thế. Điều này phù hợp với việc He và cộng sự huấn luyện được mạng 152 tầng, trong khi trong cùng bài báo, mạng thuần tích 34 tầng có sai số huấn luyện *cao hơn* mạng thuần tích 18 tầng. Về mặt nguyên lý, đây đúng là cơ chế mà cổng LSTM dùng trong miền thời gian (Mục V.), chỉ chuyển sang miền chiều sâu.

Thứ tự ưu tiên khi gặp mạng sâu không học được

(1) Xác nhận đây là vấn đề tối ưu hoá bằng cách kiểm tra sai số *huấn luyện*. (2) In bảng chuẩn gradient theo tầng. (3) Nếu triệt tiêu: đổi sang ReLU/GELU và khởi tạo He — đây là hai thay đổi rẻ nhất về chi phí tính toán, và có thể đủ. (4) Nếu vẫn chưa đủ: thêm chuẩn hoá. (5) Nếu mạng rất sâu: thêm kết nối tắt. (6) Nếu bùng nổ: cắt ngưỡng gradient trước, rồi mới xem lại khởi tạo.

VII.2. Liên hệ với tài liệu Module 4

Các chủ đề trên được phát triển đầy đủ, kèm chứng minh chi tiết và mã nguồn đã thực thi, trong sổ tay `docs/module4-deeplearning-optimization.tex`:

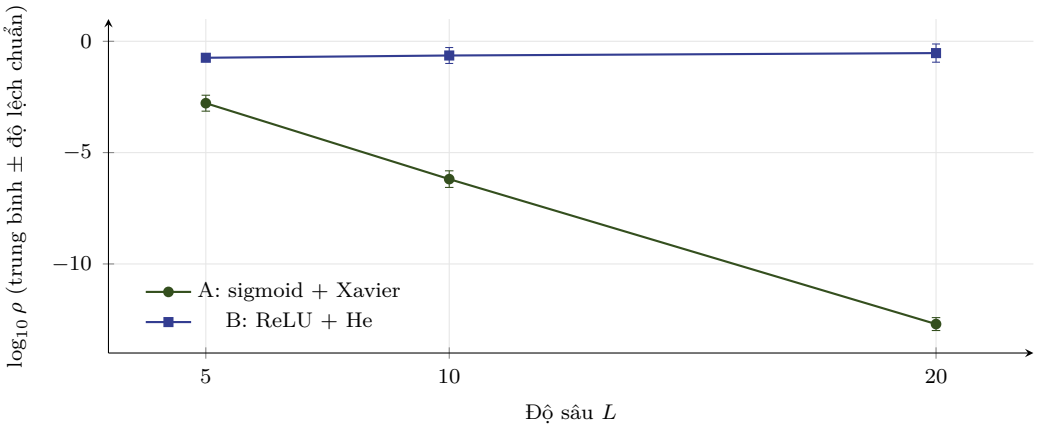
- **Mục 11** — Hàm kích hoạt và đạo hàm: họ bão hoà so với họ không bão hoà, softmax và cross-entropy.
- **Mục 13** — Triệt tiêu, bùng nổ gradient và khởi tạo trọng số: chứng minh đầy đủ định lý Xavier/Glorot và He/Kaiming.

- **Mục 14** — Điều hoà và chuẩn hoá: inverted dropout, BatchNorm, so sánh các sơ đồ chuẩn hoá.
- **Mục 18** — Kết nối tắt và dòng chảy gradient: phân tích khai triển $\mathbf{I} + \mathcal{F}'$ và ý nghĩa với mặt mất mát.

Tài liệu hiện tại đóng vai trò dẫn nhập tập trung vào *nguyên nhân*; sổ tay Module 4 bao quát toàn bộ không gian giải pháp và lý thuyết tối ưu hoá đi kèm.

A. Hình bổ sung: $\log_{10}\rho$ theo độ sâu tại bước khởi tạo

Hình 5 vẽ lại các cột $\log_{10}\rho$ của Bảng 3 dưới dạng đồ thị; dữ liệu trùng với bảng nên hình được đưa về phụ lục.



Hình 5: $\log_{10}\rho$, với $\rho = \|\mathbf{g}^{[1]}\|_F/\|\mathbf{g}^{[L]}\|_F$, theo độ sâu $L \in \{5, 10, 20\}$ của MLP rộng 64 tại bước khởi tạo, trung bình ± độ lệch chuẩn trên 5 hạt giống. Điều kiện A (sigmoid + Xavier) giảm gần tuyến tính theo L (khoảng 0.2 mỗi tầng); điều kiện B (ReLU + He) nằm gần 0 nhưng thanh sai số rộng dần theo L. Trục y là $\log_{10}\rho$ (tương đương thang log).

Tài liệu tham khảo

[1] R. Pascanu et al., "On the difficulty of training recurrent neural networks", in *Proceedings of the 30th International Conference on Machine Learning (ICML)*, 2013, pp. 1310–1318.

[2] S. Hochreiter, "Untersuchungen zu dynamischen neuronalen netzen", Diploma thesis, Technische Universität München, Munich, Germany, 1991.

[3] Y. Bengio et al., "Learning long-term dependencies with gradient descent is difficult", *IEEE Transactions on Neural Networks*, vol. 5, no. 2, pp. 157–166, 1994.

[4] X. Glorot et al., "Understanding the difficulty of training deep feedforward neural networks", in *Proceedings of the 13th International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2010, pp. 249–256.

[5] K. He et al., "Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification", in *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2015, pp. 1026–1034.

[6] S. Ioffe et al., "Batch normalization: Accelerating deep network training by reducing internal covariate shift", in *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, 2015, pp. 448–456.

[7] K. He et al., "Deep residual learning for image recognition", in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.

[8] L. Xiao et al., "Dynamical isometry and a mean field theory of CNNs: How to train 10,000-layer vanilla convolutional neural networks", in *Proceedings of the 35th International Conference on Machine Learning (ICML)*, 2018, pp. 5393–5402.

[9] G. Yang et al., "A mean field theory of batch normalization", in *International Conference on Learning Representations (ICLR)*, 2019.